Принадлежность точки тетраэдру

Командный отчёт по линейной алгебре

Участники: Розыев Атаджан, Алымов Тихон, Немировский Роман. Группа: 4.

В работе исследованы четыре способа определения положения точки относительно тетраэдра: объёмы, знаки смешанных произведений, уравнения плоскостей и барицентрические координаты. Для каждого способа приведены обоснование, ручные расчёты и проверка на Python. Все четыре метода дали одинаковые результаты на выбранных примерах. Рисунки и полный исходный код включены в этот отчёт.

1 Цель

Изучить методы векторной алгебры и аналитической геометрии для определения принадлежности точки тетраэдру. Сравнить их на одинаковых исходных данных, рассмотреть все пять положений точки и сформулировать прикладную задачу.

2 Постановка задачи

Для невырожденного тетраэдра ABCD и точки P(x; y; z) определить, находится ли P внутри, на грани, на ребре, совпадает ли с вершиной или лежит снаружи. Граница включается в тетраэдр.

3 Вершины тетраэдра и проверка корректности

Вершина	Координаты
A	(0; 0; 0)
B	(6; 0; 0)
C	(0; 6; 0)
D	(0; 0; 6)

Векторы из вершины A: AB = (6; 0; 0), AC = (0; 6; 0), AD = (0; 0; 6). Их смешанное произведение:

$$d_0 = [AB, AC, AD] = \begin{vmatrix} 6 & 0 & 0 \\ 0 & 6 & 0 \\ 0 & 0 & 6 \end{vmatrix} = 6 \cdot 6 \cdot 6 = 216.$$

$$V_0 = \frac{|d_0|}{6} = \frac{216}{6} = 36 \neq 0.$$

Следовательно, вершины не лежат в одной плоскости и тетраэдр невырожден. Все объёмы далее указаны в кубических единицах.

4 Методы решения

4.1 Метод объёмов через смешанное произведение

4.1.1 Формула объёма и основная идея

Для векторов u, v, w смешанное произведение [u, v, w] = u · (v × w) равно определителю матрицы их координат. Его модуль задаёт объём параллелепипеда. Объём тетраэдра составляет одну шестую этого объёма:

$$V(U,V,W,T) = \frac{|[UV,UW,UT]|}{6}.$$

Обозначения UV, UW и UT здесь означают векторы из точки U. Модуль обязателен: геометрический объём неотрицателен.

Заменим каждую вершину исходного тетраэдра точкой P. Введём обозначения:

$$V_1 = V(PBCD),\ V_2 = V(APCD),\ V_3 = V(ABPD),\ V_4 = V(ABCP).$$

$$S = V_1 + V_2 + V_3 + V_4.$$

Если P находится внутри, четыре тетраэдра заполняют ABCD и не пересекаются внутренними областями. На границе некоторые из них имеют нулевой объём. В обоих случаях S = V_0.

4.1.2 Обоснование критерия принадлежности

Пусть d_0 - исходное смешанное произведение, а d_1, d_2, d_3, d_4 - смешанные произведения после замены соответственно A, B, C, D точкой P, с сохранением порядка вершин. Из линейности определителя следует тождество $d_1 + d_2 + d_3 + d_4 = d_0$. Поэтому:

$$S = \frac{|d_1| + |d_2| + |d_3| + |d_4|}{6} \geq \frac{|d_0|}{6} = V_0.$$

Внутри тетраэдра ненулевые d_i имеют тот же знак, что и d_0. Если P снаружи, хотя бы одна грань отделяет её от противоположной вершины; соответствующее d_i имеет противоположный знак. В сумме появляются слагаемые обоих знаков, поэтому неравенство становится строгим.

$$S = V_0 \Leftrightarrow P \in ABCD; \qquad S > V_0 \Leftrightarrow P \notin ABCD.$$

4.1.3 Как различить положения на границе

Нулевых объёмов	Положение при S = V_0
0	Строго внутри
1	Внутри грани, вне её рёбер
2	Внутри ребра, вне его концов
3	В вершине

Нулевой объём означает, что P лежит в плоскости соответствующей грани. Эту таблицу применяют только после проверки S = V_0. Четыре нулевых объёма невозможны: исходный объём положителен.

4.1.4 Общие ручные вычисления для выбранных вершин

Пусть P(x; y; z). Выведем формулы всех четырёх объёмов один раз, затем подставим координаты тестовых точек.

Объём тетраэдра PBCD

PB = (6 − x; −y; −z), PC = (−x; 6 − y; −z), PD = (−x; −y; 6 − z).

$$d_1 = \begin{vmatrix} 6-x & -y & -z \\ -x & 6-y & -z \\ -x & -y & 6-z \end{vmatrix}.$$

Раскрываем определитель по первой строке:

$$d_1 = (6-x)((6-y)(6-z) - yz) - 6xy - 6xz$$
$$= (6-x)(36 - 6y - 6z) - 6xy - 6xz$$
$$= 216 - 36x - 36y - 36z = 36(6 - x - y - z).$$
$$V_1 = \frac{|36(6-x-y-z)|}{6} = 6|6 - x - y - z|.$$

Объём тетраэдра APCD

$$d_2 = \begin{vmatrix} x & y & z \\ 0 & 6 & 0 \\ 0 & 0 & 6 \end{vmatrix} = 36x; \quad V_2 = \frac{|36x|}{6} = 6|x|.$$

Объём тетраэдра ABPD

$$d_3 = \begin{vmatrix} 6 & 0 & 0 \\ x & y & z \\ 0 & 0 & 6 \end{vmatrix} = 36y; \quad V_3 = \frac{|36y|}{6} = 6|y|.$$

Объём тетраэдра ABCP

$$d_4 = \begin{vmatrix} 6 & 0 & 0 \\ 0 & 6 & 0 \\ x & y & z \end{vmatrix} = 36z; \quad V_4 = \frac{|36z|}{6} = 6|z|.$$

Получили формулы для ручного решения:

$$V_1 = 6|6 - x - y - z|; \;\; V_2 = 6|x|; \;\; V_3 = 6|y|; \;\; V_4 = 6|z|.$$

Эти упрощённые формулы относятся именно к выбранным вершинам. Программа использует общую формулу смешанного произведения и допускает другие невырожденные тетраэдры.

4.1.5 Ручное решение для пяти положений точки

Внутренняя точка P_1

$P_1(1; 1; 1)$. Подставим координаты в полученные формулы.

$$V_1 = 6|6 - 1 - 1 - 1| = 18;$$
$$V_2 = 6|1| = 6;\ \ V_3 = 6|1| = 6;\ \ V_4 = 6|1| = 6.$$
$$S = 18 + 6 + 6 + 6 = 36 = V_0.$$

Все четыре объёма положительны. Следовательно, P_1 находится строго внутри тетраэдра.

Точка P_2 на грани

$P_2(2; 2; 2)$.

$$V_1 = 6|6 - 2 - 2 - 2| = 0;$$
$$V_2 = 6|2| = 12;\ \ V_3 = 6|2| = 12;\ \ V_4 = 6|2| = 12.$$
$$S = 0 + 12 + 12 + 12 = 36 = V_0.$$

Нулевой только V_1. Следовательно, P_2 лежит на грани BCD, вне её рёбер.

Точка P_3 на ребре

$P_3(3; 3; 0)$.

$$V_1 = 6|6 - 3 - 3 - 0| = 0;$$
$$V_2 = 6|3| = 18;\ \ V_3 = 6|3| = 18;\ \ V_4 = 6|0| = 0.$$
$$S = 0 + 18 + 18 + 0 = 36 = V_0.$$

Нулевые V_1 и V_4 соответствуют граням BCD и ABC. Их пересечение - ребро BC.
Следовательно, P_3 лежит на ребре BC, вне его концов.

Точка P_4 в вершине

$P_4(0; 0; 0)$.

$$V_1 = 6|6 - 0 - 0 - 0| = 36;$$
$$V_2 = 6|0| = 0;\ \ V_3 = 6|0| = 0;\ \ V_4 = 6|0| = 0.$$
$$S = 36 + 0 + 0 + 0 = 36 = V_0.$$

Три объёма равны нулю. Точка P_4 совпадает с вершиной A.

Внешняя точка P_5

$P_5(3; 3; 3)$.

$$V_1 = 6|6 - 3 - 3 - 3| = 6|-3| = 18;$$
$$V_2 = 6|3| = 18;\ \ V_3 = 6|3| = 18;\ \ V_4 = 6|3| = 18.$$
$$S = 18 + 18 + 18 + 18 = 72 > 36 = V_0.$$

Сумма объёмов больше исходного объёма. Следовательно, P_5 находится снаружи тетраэдра.

4.1.6 Результаты и алгоритм проверки

В таблице сопоставлены ручные результаты и положения, определённые программой. Исходный объём $V_0 = 36$ во всех строках.

Точка	Координаты	V_1	V_2	V_3	V_4	S	Результат
P_1	(1; 1; 1)	18	6	6	6	36	Внутри
P_2	(2; 2; 2)	0	12	12	12	36	Грань BCD
P_3	(3; 3; 0)	0	18	18	0	36	Ребро BC
P_4	(0; 0; 0)	36	0	0	0	36	Вершина A
P_5	(3; 3; 3)	18	18	18	18	72	Снаружи

Дополнительная проверка внешней точки

Возьмём Q(−1; 0; 0). Эта точка лежит на продолжении прямой AB, за пределами ребра AB.

$$V_1 = 6|6 - (-1) - 0 - 0| = 42; \quad V_2 = 6|-1| = 6;$$

$$V_3 = 0; \quad V_4 = 0; \quad S = 42 + 6 = 48 > 36.$$

Несмотря на два нулевых объёма, Q находится снаружи. Пример подтверждает необходимость сначала сравнивать S и V_0.

Последовательность действий программы

1. Вычислить V_0. Если $V_0 = 0$, сообщить о вырожденном тетраэдре.

2. Вычислить V_1, V_2, V_3, V_4 и их сумму S.

3. Сравнить S с V_0 с учётом численного допуска. При несовпадении вернуть результат «снаружи».

4. При совпадении подсчитать объёмы, близкие к нулю, и определить положение по таблице из пункта 4.1.3.

Численный допуск

В точной математике используются равенства. При вычислениях с вещественными числами возможны ошибки округления. В программе принят относительный допуск $\varepsilon = 10^{-9}$ и порог $\delta = \varepsilon V_0$:

$$|S - V_0| \le \delta; \qquad V_i \le \delta \Rightarrow V_i \text{ считается нулевым.}$$

Такой порог масштабируется вместе с объёмом. Точки очень близко к границе могут быть отнесены к ней в пределах допуска. Для почти вырожденных тетраэдров и координат с большим разбросом масштабов может потребоваться более точная арифметика.

Для всех пяти основных точек и дополнительной точки Q результаты программы совпали с ручными вычислениями.

4.1.7 Реализация на Python

Функции subtract, mixed и volume вычисляют векторы, смешанное произведение и объём; classify_point определяет положение. Полный код всех четырёх методов и рисунков находится в приложении А.

4.1.8 Запуск программы и общие данные

Для запуска скопируйте части листинга приложения А по порядку в один файл project.py в кодировке UTF-8. Сохраните отступы; заголовки подразделов в код не включайте.

Установка и запуск

Для графических окон нужна библиотека Matplotlib. Команды выполняются в папке с файлом программы:

```
python -m pip install matplotlib
python project.py
```

Пять окон показывают общий вид, объёмы, знаки, плоскости и барицентрические веса. Во втором окне есть переключатель точек. Трёхмерные модели вращаются мышью.

Дополнительные режимы запуска:

```
python project.py --point 1 2 1
python project.py --no-plot
python project.py --no-plot --save-dir images
```

Первая команда добавляет P(1; 2; 1). Вторая выполняет только расчёты без Matplotlib. Третья сохраняет пять PNG без открытия окон; ей Matplotlib нужен. Все рисунки уже встроены в раздел 7.

Единый интерфейс четырёх методов

Метод	Функция	Состояние
1	method_volumes	Реализован
2	method_signs	Реализован
3	method_planes	Реализован
4	method_alternative	Реализован

Функции получают (a, b, c, d, p, eps=1e-9) и возвращают положение точки. Список METHODS вызывает все четыре метода. Функции classify_* возвращают также промежуточные значения.

Общие данные заданы в VERTICES и TEST_POINTS. По заданию примеры выбираются самостоятельно; input() не обязателен. Данные можно изменить в этих блоках или добавить точку через --point.

4.1.9 Вывод по методу объёмов

Метод определяет принадлежность по равенству $S = V_0$, после чего по числу нулевых объёмов различаются положения на границе. Для P_1–P_5 и Q ручные и программные результаты совпали. Обработка одной точки требует O(1) времени и памяти; при N точках время составляет O(N).

4.2 Метод знаков смешанных произведений

4.2.1 Основная идея

Каждая грань тетраэдра лежит в плоскости, которая делит пространство на два полупространства. Тетраэдр является пересечением четырёх полупространств, содержащих его противоположные вершины. Поэтому положение точки P можно проверить локально: для каждой грани достаточно установить, находится ли P с той же стороны от её плоскости, что и противоположная вершина.

Сторону плоскости удобно определять по знаку смешанного произведения. Для грани UVW и точки X величина [UV, UW, UX] положительна с одной стороны плоскости UVW, отрицательна с другой и равна нулю в самой плоскости. Сравнивая знаки s(P) = [UV, UW, UP] и s(T) = [UV, UW, UT], где T - противоположная вершина, получаем критерий принадлежности без вычисления объёмов и их суммы.

Если для каждой грани знак s(P) совпадает со знаком s(T) или s(P) = 0, точка P принадлежит тетраэдру. Если хотя бы для одной грани знаки противоположны, P лежит снаружи. Нулевое s(P) означает принадлежность плоскости грани; само по себе оно ещё не доказывает принадлежность треугольной грани.

4.2.2 Математическое обоснование

Зафиксируем грань UVW и противоположную вершину T. Векторы UV, UW и UT не компланарны, поскольку тетраэдр невырожден. Поэтому

$$s(T) = [UV, UW, UT] \neq 0.$$

Функция f(X) = [UV, UW, UX] аффинна по координатам X, а уравнение f(X) = 0 задаёт плоскость UVW. В каждом открытом полупространстве знак f постоянен. Поскольку f(T) ≠ 0, совпадение знаков f(P) и f(T) либо равенство f(P) = 0 означает, что P лежит в замкнутом полупространстве, содержащем T. Через отношение это записывается так:

$$P \in H(UVW) \Leftrightarrow \frac{s(P)}{s(T)} \geq 0.$$

Тетраэдр ABCD является выпуклой оболочкой своих вершин, поэтому он равен пересечению четырёх полупространств, ограниченных плоскостями граней и содержащих противоположные вершины. Следовательно,

$$P \in ABCD \Leftrightarrow \frac{s(P)}{s(T)} \geq 0 \text{ для всех четырёх граней.}$$

Для выбранных вершин пары «грань - противоположная вершина» и соответствующие произведения приведены в таблице.

Грань	Вершина T	s(P)	s(T)	r = s(P)/s(T)
ABC	D	[AB, AC, AP]	[AB, AC, AD]	r(D) = λ(D)
ABD	C	[AB, AD, AP]	[AB, AD, AC]	r(C) = λ(C)
ACD	B	[AC, AD, AP]	[AC, AD, AB]	r(B) = λ(B)
BCD	A	[BC, BD, BP]	[BC, BD, BA]	r(A) = λ(A)

Отношения r = s(P)/s(T) являются барицентрическими координатами соответствующих противоположных вершин T. Их сумма равна 1. В методе 4 эти координаты вычисляются независимо, решением системы уравнений методом Гаусса.

После проверки принадлежности число нулевых отношений определяет положение: 0 - внутри, 1 - на грани, 2 - на ребре, 3 - в вершине. Четыре нуля невозможны, поскольку сумма отношений равна 1. Внешняя точка тоже может иметь нулевые отношения, поэтому сначала исключают отрицательные значения; пример - точка Q.

В программе используется безразмерное отношение r = s(P)/s(T) и допуск $\varepsilon = 10^{-9}$: при $|r| \le \varepsilon$ значение считают нулевым, при $r < -\varepsilon$ - отрицательным. Почти вырожденный тетраэдр делает вычисления чувствительными к округлению и может потребовать повышенной точности.

4.2.3 Общие ручные вычисления для выбранных вершин

Для вершин A(0; 0; 0), B(6; 0; 0), C(0; 6; 0), D(0; 0; 6) вычислим векторные произведения, входящие в смешанные произведения. Все векторы граней берутся из первой вершины грани:

$$\mathrm{AB} \times \mathrm{AC} = (0;\ 0;\ 36),\quad \mathrm{AB} \times \mathrm{AD} = (0;\ -36;\ 0),$$
$$\mathrm{AC} \times \mathrm{AD} = (36;\ 0;\ 0),\quad \mathrm{BC} \times \mathrm{BD} = (36;\ 36;\ 36).$$

Пусть P(x; y; z). Тогда

$$s_1(\mathrm{P}) = [\mathrm{AB}, \mathrm{AC}, \mathrm{AP}] = 36z,\quad s_1* = [\mathrm{AB}, \mathrm{AC}, \mathrm{AD}] = 216;$$
$$s_2(\mathrm{P}) = [\mathrm{AB}, \mathrm{AD}, \mathrm{AP}] = -36y,\quad s_2* = [\mathrm{AB}, \mathrm{AD}, \mathrm{AC}] = -216;$$
$$s_3(\mathrm{P}) = [\mathrm{AC}, \mathrm{AD}, \mathrm{AP}] = 36x,\quad s_3* = [\mathrm{AC}, \mathrm{AD}, \mathrm{AB}] = 216;$$
$$s_4(\mathrm{P}) = [\mathrm{BC}, \mathrm{BD}, \mathrm{BP}] = 36(x + y + z - 6),\quad s_4* = [\mathrm{BC}, \mathrm{BD}, \mathrm{BA}] = -216.$$

Звёздочкой отмечены произведения для противоположной вершины. Все четыре значения s_i* отличны от нуля, поэтому критерий знаков применим. Соответствующие отношения равны

$$r(\mathrm{D}) = s_1/s_1* = \frac{z}{6},\quad r(\mathrm{C}) = s_2/s_2* = \frac{y}{6},$$
$$r(\mathrm{B}) = s_3/s_3* = \frac{x}{6},\quad r(\mathrm{A}) = s_4/s_4* = \frac{6 - x - y - z}{6}.$$

Это барицентрические координаты точки P, отвечающие вершинам D, C, B и A соответственно. Их сумма равна единице: r(A) + r(B) + r(C) + r(D) = 1. Для внутренней точки все четыре координаты положительны.

4.2.4 Ручное решение для тестовых точек

Во всех расчётах $s^* = (216; -216; 216; -216)$. Порядок компонент: s_1 - грань ABC, s_2 - ABD, s_3 - ACD, s_4 - BCD.

Точка P_1 - внутри

$P_1(1; 1; 1)$.

$$s_1 = 36 \cdot 1 = 36; \quad s_2 = -36 \cdot 1 = -36; \quad s_3 = 36 \cdot (1) = 36.$$
$$s_4 = 36 \cdot ((1) + 1 + 1 - 6) = -108.$$
$$s = (36; -36; 36; -108), \quad r = (1/6; 1/6; 1/6; 1/2).$$

Все четыре произведения имеют те же знаки, что и соответствующие s_i^*: плюс с плюсом, минус с минусом. Нулевых значений нет, все отношения положительны, их сумма равна 1. Следовательно, P_1 лежит строго внутри тетраэдра.

Точка P_2 - на грани

$P_2(2; 2; 2)$.

$$s_1 = 36 \cdot 2 = 72; \quad s_2 = -36 \cdot 2 = -72; \quad s_3 = 36 \cdot (2) = 72.$$
$$s_4 = 36 \cdot ((2) + 2 + 2 - 6) = 0.$$
$$s = (72; -72; 72; 0), \quad r = (1/3; 1/3; 1/3; 0).$$

Первые три знака совпадают, а $s_4 = 0$. Один нуль означает, что P_2 лежит в плоскости грани BCD. Остальные отношения положительны, поэтому точка находится внутри треугольника BCD, а не на его ребре. Результат - грань BCD.

Точка P_3 - на ребре

$P_3(3; 3; 0)$.

$$s_1 = 36 \cdot 0 = 0; \quad s_2 = -36 \cdot 3 = -108; \quad s_3 = 36 \cdot (3) = 108.$$
$$s_4 = 36 \cdot ((3) + 3 + 0 - 6) = 0.$$
$$s = (0; -108; 108; 0), \quad r = (0; 1/2; 1/2; 0).$$

Нулевые s_1 и s_4 соответствуют граням ABC и BCD. Их пересечение - ребро BC. Два других отношения положительны, значит P_3 лежит внутри ребра BC, а не в его концах. Результат - ребро BC.

Точка P_4 - в вершине

$P_4(0; 0; 0)$.

$$s_1 = 36 \cdot 0 = 0;\ \ s_2 = -36 \cdot 0 = 0;\ \ s_3 = 36 \cdot (0) = 0.$$
$$s_4 = 36 \cdot ((0) + 0 + 0 - 6) = -216.$$
$$s = (0; 0; 0; -216),\ \ r = (0; 0; 0; 1).$$

Три нуля соответствуют граням ABC, ABD и ACD, пересечение которых - вершина A. Четвёртое отношение равно 1 и положительно. Следовательно, P_4 совпадает с вершиной A.

Точка P_5 - снаружи

$P_5(3; 3; 3)$.

$$s_1 = 36 \cdot 3 = 108;\ \ s_2 = -36 \cdot 3 = -108;\ \ s_3 = 36 \cdot (3) = 108.$$
$$s_4 = 36 \cdot ((3) + 3 + 3 - 6) = 108.$$
$$s = (108; -108; 108; 108),\ \ r = (1/2; 1/2; 1/2; -1/2).$$

Для грани BCD произведение $s_4 = 108$ положительно, а $s_4^* = -216$ отрицательно. Знаки разные, $r_4 = -1/2 < 0$. Следовательно, P_5 находится снаружи, по другую сторону плоскости BCD от вершины A.

Точка Q - снаружи, несмотря на два нуля

$Q(-1; 0; 0)$.

$$s_1 = 36 \cdot 0 = 0;\ \ s_2 = -36 \cdot 0 = 0;\ \ s_3 = 36 \cdot (-1) = -36.$$
$$s_4 = 36 \cdot ((-1) + 0 + 0 - 6) = -252.$$
$$s = (0; 0; -36; -252),\ \ r = (0; 0; -1/6; 7/6).$$

Для грани ACD произведение $s_3 = -36$, а $s_3^* = 216$: знаки разные, $r_3 = -1/6$. Точка лежит снаружи за гранью ACD. Два нулевых произведения s_1 и s_2 не означают ребро: сначала нужно исключить отрицательные отношения. Этот пример показывает, почему порядок проверок в алгоритме важен. Сумма отношений по-прежнему равна 1: $-1/6 + 7/6 = 1$.

Сводка ручных вычислений приведена в таблице.

Точка	s_1	s_2	s_3	s_4	Результат
P_1	36	−36	36	−108	Внутри
P_2	72	−72	72	0	Грань BCD
P_3	0	−108	108	0	Ребро BC
P_4	0	0	0	−216	Вершина A
P_5	108	−108	108	108	Снаружи (за BCD)
Q	0	0	−36	−252	Снаружи (за ACD)

4.2.5 Решение с помощью кода

В приложении А функции classify_signs и method_signs реализуют метод 2. Первая возвращает положение, s(P), s(T) и отношения r; вторая возвращает только положение для общей таблицы. Векторы вычисляются по введённым вершинам, поэтому формулы подходят для произвольного невырожденного тетраэдра.

Алгоритм:

1. Для каждой из четырёх граней вычислить s(P) и s(T) по формуле смешанного произведения.

2. Проверить $s(T) \neq 0$. Нулевое значение означает вырожденный тетраэдр.

3. Вычислить $r = s(P)/s(T)$. Если хотя бы одно $r < -\varepsilon$, вернуть «снаружи».

4. Иначе подсчитать число значений $|r| \leq \varepsilon$ и по нему выбрать «внутри», «на грани», «на ребре» или «в вершине».

В приведённой реализации на каждую точку вычисляются восемь смешанных произведений. Четыре из них, s(T), зависят только от тетраэдра; при пакетной обработке их можно вычислить один раз и кэшировать, после чего на точку останется четыре произведения. Сложность по времени - O(1) на точку, по памяти - O(1).

4.2.6 Проверка результатов

Запуск полной программы из приложения А воспроизводит значения s и r из пункта 4.2.4: P_1 - внутри, P_2 - на грани, P_3 - на ребре, P_4 - в вершине, P_5 и Q - снаружи. Числа округляются только при печати; классификация использует значения без этого округления.

Все шесть результатов совпали с ручными вычислениями. Общая проверка четырёх методов и дополнительные тесты описаны в пункте 5.4. Эти тесты дополняют математическое обоснование корректности.

4.2.7 Вывод по методу

Метод знаков корректен для любого невырожденного тетраэдра и не требует вычисления объёмов. Его главное преимущество - прямая геометрическая интерпретация: каждая проверка отвечает за одну грань, поэтому при внешнем положении сразу видно, какая грань нарушена. Нулевые отношения естественно различают грань, ребро и вершину, а сами отношения являются барицентрическими координатами и в сумме дают 1, что удобно для контроля.

Ограничения связаны с вычислением s(T) и с почти вырожденными тетраэдрами: если s(T) мало, отношение чувствительно к ошибкам округления. В приведённой реализации на точку приходится восемь смешанных произведений; после кэширования s(T) - четыре, то есть столько же, сколько в методе объёмов. Для задачи классификации метод 2 удобен и универсален; для пакетной обработки большого числа точек он сопоставим по стоимости с методом объёмов, но даёт более подробную диагностику границы.

4.3 Метод уравнений плоскостей

4.3.1 Основная идея

Каждая грань тетраэдра лежит в своей плоскости, а плоскость делит пространство на два полупространства. Тетраэдр ABCD - это общая часть четырёх полупространств: для каждой грани берётся то, в котором лежит противоположная вершина. Поэтому для каждой грани достаточно выяснить, по какую сторону от её плоскости находится точка P.

Сторона определяется знаком левой части уравнения плоскости f(x; y; z) = 0. Если значение f(P) имеет тот же знак, что и значение f(t) в противоположной вершине t, то P лежит с той же стороны, что и t. Если знаки разные, грань отделяет P от тетраэдра, и P снаружи. Если f(P) = 0, точка лежит в плоскости грани. Число таких нулей по четырём граням различает внутреннюю точку, грань, ребро и вершину. Число нулей используют для классификации границы только после проверки отсутствия противоположных знаков.

4.3.2 Математическое обоснование

Уравнение плоскости грани. Пусть грань задана вершинами u, v, w, а X(x; y; z) - произвольная точка. Нормаль к плоскости грани - векторное произведение $n = (v - u) \times (w - u)$. Оно не равно нулю, потому что вершины грани не лежат на одной прямой. Условие $n \cdot (X - u) = 0$ даёт уравнение плоскости:

$$f(x; y; z) = n_1x + n_2y + n_3z + k = 0, \quad k = -(n_1u_1 + n_2u_2 + n_3u_3).$$

Та же функция записывается определителем, так как $n \cdot (X - u) = [X - u, v - u, w - u]$:

$$f(X) = \begin{vmatrix} x - u_1 & y - u_2 & z - u_3 \\ v_1 - u_1 & v_2 - u_2 & v_3 - u_3 \\ w_1 - u_1 & w_2 - u_2 & w_3 - u_3 \end{vmatrix}.$$

Для грани ABC это f(P) = [AB, AC, AP] - то же смешанное произведение, что и в методах 1 и 2. Поэтому $|f(P)| = 6 \cdot V(ABCP)$: значение уравнения плоскости с точностью до знака равно шестикратному объёму соответствующего тетраэдра.

Знак значения и сторона плоскости. Функция f аффинна, поэтому для точек отрезка XY выполняется $f((1 - s)X + sY) = (1 - s)f(X) + s \cdot f(Y)$, $0 \le s \le 1$. Если f(X) и f(Y) имеют разные знаки, на отрезке есть точка с f = 0: отрезок пересекает плоскость, и точки лежат по разные стороны. Если знаки одинаковы, нуля нет, и точки лежат по одну сторону. Вершина t не лежит в плоскости противоположной грани, потому что тетраэдр невырожден; значит, $f(t) \neq 0$.

Критерий принадлежности. Нумерация граней согласована с методом объёмов: f_1 - плоскость BCD (противоположная вершина $t_1 = A$), f_2 - ACD ($t_2 = B$), f_3 - ABD ($t_3 = C$), f_4 - ABC ($t_4 = D$). Введём отношения

$$g_i = \frac{f_i(P)}{f_i(t_i)}, \quad i = 1, 2, 3, 4.$$

Отношение не зависит от масштаба и направления нормали: при замене n на −n знаки f(P) и f(t) меняются одновременно. Условие $g_i \ge 0$ означает, что P лежит в замкнутом полупространстве, содержащем t_i. Так как тетраэдр - пересечение четырёх таких полупространств, получаем критерий:

$$P \in ABCD \iff g_1 \ge 0, \quad g_2 \ge 0, \quad g_3 \ge 0, \quad g_4 \ge 0.$$

Если хотя бы одно g_i отрицательно, P находится снаружи. Числа g_i - барицентрические координаты точки: каждая g_i - аффинная функция, равная 1 в вершине t_i и 0 в трёх других вершинах (они лежат в плоскости i-й грани), поэтому $g_1 + g_2 + g_3 + g_4 = 1$ и $P = g_1A + g_2B + g_3C + g_4D$. Кроме того, $|g_i| = V_i / V_0$, где V_1–V_4 - объёмы из метода 1, так как $|f_i(P)| = 6V_i$, а $|f_i(t_i)| = 6V_0$. Это связывает метод 3 с методом 1 и даёт проверку: сумма g_i всегда равна 1.

Различение положений. Если все $g_i \geq 0$, положение на границе определяется числом нулевых отношений. Ребро - пересечение плоскостей двух граней, вершина - пересечение плоскостей трёх граней. Четыре нуля невозможны, так как сумма g_i равна 1.

Нулевых g_i	Положение при всех $g_i \geq 0$
0	Строго внутри
1	Внутри грани, вне её рёбер
2	Внутри ребра, вне его концов
3	В вершине

Таблицу применяют только после проверки знаков. Точка с двумя нулевыми g_i и одним отрицательным (как Q ниже) лежит на прямой, содержащей ребро, но вне самого ребра, то есть снаружи.

Уравнения плоскостей для выбранных вершин. Векторы из вершин: AB = (6; 0; 0), AC = (0; 6; 0), AD = (0; 0; 6), BC = (−6; 6; 0), BD = (−6; 0; 6). Нормали и коэффициенты k:

BCD (u = B): $n = BC \times BD = (6\cdot6 - 0\cdot0;\ 0\cdot(-6) - (-6)\cdot6;\ (-6)\cdot0 - 6\cdot(-6)) = (36; 36; 36)$, $k = -36\cdot6 = -216$.

ACD (u = A): $n = AC \times AD = (6\cdot6 - 0\cdot0;\ 0\cdot0 - 0\cdot6;\ 0\cdot0 - 6\cdot0) = (36; 0; 0)$, $k = 0$.

ABD (u = A): $n = AB \times AD = (0\cdot6 - 0\cdot0;\ 0\cdot0 - 6\cdot6;\ 6\cdot0 - 0\cdot0) = (0; -36; 0)$, $k = 0$.

ABC (u = A): $n = AB \times AC = (0\cdot0 - 0\cdot6;\ 0\cdot0 - 6\cdot0;\ 6\cdot6 - 0\cdot0) = (0; 0; 36)$, $k = 0$.

№	Грань	Нормаль n	Уравнение $f_i = 0$	$f_i(t_i)$	g_i
1	BCD	(36; 36; 36)	$36x + 36y + 36z - 216$	$f_1(A) = -216$	$(6 - x - y - z) / 6$
2	ACD	(36; 0; 0)	$36x$	$f_2(B) = 216$	$x / 6$
3	ABD	(0; −36; 0)	$-36y$	$f_3(C) = -216$	$y / 6$
4	ABC	(0; 0; 36)	$36z$	$f_4(D) = 216$	$z / 6$

Сокращённые уравнения граней: $x + y + z = 6$, $x = 0$, $y = 0$, $z = 0$. Следовательно, для выбранных вершин тетраэдр задаётся системой $x \geq 0$, $y \geq 0$, $z \geq 0$, $x + y + z \leq 6$. Формулы g_i согласуются с методом 1: например, $V_2 = 6|x|$, $V_0 = 36$, откуда $|g_2| = |x| / 6$. Эти упрощённые формулы относятся именно к выбранным вершинам; программа использует общую формулу с векторным произведением.

4.3.3 Решение вручную

Подставим координаты каждой точки в f_1, f_2, f_3, f_4 и сравним знак с значением в противоположной вершине: $f_1(A) = -216$, $f_2(B) = 216$, $f_3(C) = -216$, $f_4(D) = 216$.

Внутренняя точка $P_1(1; 1; 1)$.

$$f_1 = 36(1 + 1 + 1 - 6) = -108,\ g_1 = \frac{-108}{-216} = 1/2 > 0$$

$$f_2 = 36 \cdot 1 = 36,\ g_2 = \frac{36}{216} = 1/6 > 0$$

$$f_3 = -36 \cdot 1 = -36,\ g_3 = \frac{-36}{-216} = 1/6 > 0$$

$$f_4 = 36 \cdot 1 = 36,\ g_4 = \frac{36}{216} = 1/6 > 0$$

$$g_1 + g_2 + g_3 + g_4 = 1/2 + 1/6 + 1/6 + 1/6 = 1.$$

Все четыре отношения положительны, нулей нет. Следовательно, P_1 находится строго внутри тетраэдра.

Точка $P_2(2; 2; 2)$ на грани.

$$f_1 = 36(2 + 2 + 2 - 6) = 0,\ g_1 = \frac{0}{-216} = 0$$

$$f_2 = 36 \cdot 2 = 72,\ g_2 = \frac{72}{216} = 1/3 > 0$$

$$f_3 = -36 \cdot 2 = -72,\ g_3 = \frac{-72}{-216} = 1/3 > 0$$

$$f_4 = 36 \cdot 2 = 72,\ g_4 = \frac{72}{216} = 1/3 > 0$$

$$g_1 + g_2 + g_3 + g_4 = 0 + 1/3 + 1/3 + 1/3 = 1.$$

Отношение $g_1 = 0$: P_2 лежит в плоскости BCD. Остальные три положительны, нулевое значение одно. Следовательно, P_2 лежит на грани BCD, вне её рёбер.

Точка $P_3(3; 3; 0)$ на ребре.

$$f_1 = 36(3 + 3 + 0 - 6) = 0,\ g_1 = \frac{0}{-216} = 0$$

$$f_2 = 36 \cdot 3 = 108,\ g_2 = \frac{108}{216} = 1/2 > 0$$

$$f_3 = -36 \cdot 3 = -108,\ g_3 = \frac{-108}{-216} = 1/2 > 0$$

$$f_4 = 36 \cdot 0 = 0,\ g_4 = \frac{0}{216} = 0$$

$$g_1 + g_2 + g_3 + g_4 = 0 + 1/2 + 1/2 + 0 = 1.$$

Нулевые g_1 и g_4 означают, что P_3 лежит в плоскостях BCD и ABC. Их пересечение - ребро BC. Отношения $g_2 = g_3 = 1/2$ положительны. Следовательно, P_3 лежит на ребре BC, вне его концов.

Точка $P_4(0; 0; 0)$ в вершине.

$$f_1 = 36(0 + 0 + 0 - 6) = -216,\ g_1 = \frac{-216}{-216} = 1 > 0$$

$$f_2 = 36 \cdot 0 = 0,\ g_2 = \frac{0}{216} = 0$$

$$f_3 = -36 \cdot 0 = 0,\ g_3 = \frac{0}{-216} = 0$$

$$f_4 = 36 \cdot 0 = 0,\ g_4 = \frac{0}{216} = 0$$

$$g_1 + g_2 + g_3 + g_4 = 1 + 0 + 0 + 0 = 1.$$

Нулевые g_2, g_3, g_4: P_4 лежит в плоскостях ACD, ABD и ABC, которые пересекаются в вершине A. Следовательно, P_4 совпадает с вершиной A.

Внешняя точка $P_5(3; 3; 3)$.

$$f_1 = 36(3 + 3 + 3 - 6) = 108,\ g_1 = \frac{108}{-216} = -1/2 < 0$$

$$f_2 = 36 \cdot 3 = 108,\ g_2 = \frac{108}{216} = 1/2 > 0$$

$$f_3 = -36 \cdot 3 = -108,\ g_3 = \frac{-108}{-216} = 1/2 > 0$$

$$f_4 = 36 \cdot 3 = 108,\ g_4 = \frac{108}{216} = 1/2 > 0$$

$$g_1 + g_2 + g_3 + g_4 = (-1/2) + 1/2 + 1/2 + 1/2 = 1.$$

Для грани BCD значение $f_1(P_5) = 108$ положительно, а $f_1(A) = -216$ отрицательно: знаки разные, $g_1 = -1/2 < 0$. Плоскость BCD разделяет P_5 и вершину A. Остальные три отношения положительны, но одного отрицательного достаточно: P_5 находится снаружи.

Дополнительная внешняя точка $Q(-1; 0; 0)$.

$$f_1 = 36((-1) + 0 + 0 - 6) = -252,\ g_1 = \frac{-252}{-216} = 7/6 > 0$$

$$f_2 = 36 \cdot (-1) = -36,\ g_2 = \frac{-36}{216} = -1/6 < 0$$

$$f_3 = -36 \cdot 0 = 0,\ g_3 = \frac{0}{-216} = 0$$

$$f_4 = 36 \cdot 0 = 0,\ g_4 = \frac{0}{216} = 0$$

$$g_1 + g_2 + g_3 + g_4 = 7/6 + (-1/6) + 0 + 0 = 1.$$

Для грани ACD значение $f_2(Q) = -36$ отрицательно, а $f_2(B) = 216$ положительно: $g_2 = -1/6 < 0$. Нулевые g_3 и g_4 не делают точку принадлежащей тетраэдру: Q лежит на прямой AB за вершиной A, то есть снаружи.

Сводка. Итоги вычислений сведены в таблицы. Первая содержит значения f_i, вторая - отношения g_i.

Точка	Координаты	f_1	f_2	f_3	f_4	Нулей	Результат
P_1	(1; 1; 1)	−108	36	−36	36	0	Внутри
P_2	(2; 2; 2)	0	72	−72	72	1	Грань BCD
P_3	(3; 3; 0)	0	108	−108	0	2	Ребро BC
P_4	(0; 0; 0)	−216	0	0	0	3	Вершина A
P_5	(3; 3; 3)	108	108	−108	108	0	Снаружи
Q	(−1; 0; 0)	−252	−36	0	0	2	Снаружи

Точка	g_1	g_2	g_3	g_4	Σg_i	Условие
P_1	1/2	1/6	1/6	1/6	1	$\min g_i \geq 0$
P_2	0	1/3	1/3	1/3	1	$\min g_i \geq 0$
P_3	0	1/2	1/2	0	1	$\min g_i \geq 0$
P_4	1	0	0	0	1	$\min g_i \geq 0$
P_5	−1/2	1/2	1/2	1/2	1	есть $g_i < 0$
Q	7/6	−1/6	0	0	1	есть $g_i < 0$

Во всех шести строках сумма g_i равна 1, что подтверждает правильность вычислений.

4.3.4 Решение с помощью кода

В приложении А приведены cross, plane_through, plane_value, classify_planes и method_planes. Сначала строятся четыре уравнения плоскостей, затем вычисляются отношения $g_i = f_i(P)/f_i(t)$. Функция method_planes возвращает только текст положения, а classify_planes - также четыре отношения.

Ход алгоритма. Для каждой грани программа находит нормаль и значение f(t) в противоположной вершине; нулевое f(t) означало бы вырожденный тетраэдр. Затем считается $g_i = f(P) / f(t)$. Если хотя бы одно $g_i < -\varepsilon$, возвращается «снаружи». Иначе подсчитываются g_i с $|g_i| \le \varepsilon$, и по их числу определяется «внутри», «на грани», «на ребре» или «в вершине».

Численный допуск. Допуск $\varepsilon = 10^{-9}$ относится к безразмерному отношению g_i, поэтому не зависит от масштаба координат и от длины нормали. Это удобнее, чем сравнивать f(P) с абсолютным порогом: коэффициенты плоскости растут вместе с размерами тетраэдра. Для почти вырожденных фигур и очень разных масштабов координат может понадобиться более точная арифметика.

4.3.5 Проверка результатов

Запуск кода приложения А воспроизводит таблицы из пункта 4.3.3. Результаты: P_1 - внутри, P_2 - на грани, P_3 - на ребре, P_4 - в вершине, P_5 и Q - снаружи. В разделе 7.4 показаны плоскости и положение точек относительно каждой из них.

Для всех шести точек отношения и классификация совпали с ручным решением. На 80 дополнительных геометрических случаях все четыре метода также дали ожидаемые результаты; состав тестов указан в пункте 5.4.

4.3.6 Вывод по методу

Метод уравнений плоскостей корректно определил все пять положений и дополнительную внешнюю точку Q. Критерий - четыре проверки знака; положение на границе определяется числом нулевых отношений g_i. Метод прозрачен геометрически: для выбранных вершин тетраэдр записывается неравенствами $x \ge 0$, $y \ge 0$, $z \ge 0$, $x + y + z \le 6$, и ручная проверка сводится к подстановке координат.

Отношения g_i совпадают с барицентрическими координатами и связаны с методом 1 равенством $|g_i| = V_i / V_0$, поэтому метод не противоречит методу объёмов, а даёт ту же информацию вместе со знаком. Ограничения: нужно составить четыре уравнения плоскостей; знак f надо сравнивать со знаком f(t), потому что нормаль может смотреть как внутрь, так и наружу; допуск следует задавать для отношения g_i, а не для f(P).

По числу операций: если коэффициенты плоскостей вычислены заранее, для одной точки нужны четыре значения f, каждое - три умножения и три сложения. В методе 1 для каждой точки считаются четыре смешанных произведения с предварительным вычитанием векторов. Замеры времени не проводились, поэтому вывод относится только к числу операций. В общей функции method_planes плоскости пересчитываются при каждом вызове ради единого интерфейса. Схема метода - в разделе 7.4.

4.4 Метод барицентрических координат

4.4.1 Основная идея и система уравнений

Представим точку P как взвешенную сумму четырёх вершин. Коэффициенты этой суммы называются барицентрическими координатами. Их сумма равна 1; отрицательные коэффициенты допускаются для внешних точек.

$$\mathrm{P} = \lambda_\mathrm{A}\mathrm{A} + \lambda_\mathrm{B}\mathrm{B} + \lambda_\mathrm{C}\mathrm{C} + \lambda_\mathrm{D}\mathrm{D}, \quad \lambda_\mathrm{A} + \lambda_\mathrm{B} + \lambda_\mathrm{C} + \lambda_\mathrm{D} = 1.$$

Исключим λ_A и перенесём A в левую часть. Получим систему из трёх уравнений с тремя неизвестными λ_B, λ_C и λ_D:

$$\lambda_\mathrm{B}(\mathrm{B} - \mathrm{A}) + \lambda_\mathrm{C}(\mathrm{C} - \mathrm{A}) + \lambda_\mathrm{D}(\mathrm{D} - \mathrm{A}) = \mathrm{P} - \mathrm{A}.$$

$$\mathrm{M} = (\mathrm{B} - \mathrm{A}, \mathrm{C} - \mathrm{A}, \mathrm{D} - \mathrm{A}), \quad \mathrm{M} \cdot (\lambda_\mathrm{B}; \lambda_\mathrm{C}; \lambda_\mathrm{D})^\mathrm{T} = \mathrm{P} - \mathrm{A}.$$

Векторы AB, AC и AD являются столбцами матрицы M. Её определитель равен смешанному произведению d_0. Для невырожденного тетраэдра $\det \mathrm{M} \neq 0$, поэтому система имеет единственное решение. После решения находим $\lambda_A = 1 - \lambda_B - \lambda_C - \lambda_D$. В программе система решается методом Гаусса с выбором наибольшего по модулю элемента в текущем столбце.

4.4.2 Обоснование критерия

Тетраэдр - выпуклая оболочка своих вершин. По определению выпуклой оболочки точка принадлежит ему тогда и только тогда, когда представляется суммой вершин с неотрицательными коэффициентами, сумма которых равна 1. Единственность решения системы означает, что для проверки достаточно найденных четырёх координат.

$$\mathrm{P} \in \mathrm{ABCD} \Leftrightarrow \lambda_\mathrm{A} \geq 0, \lambda_\mathrm{B} \geq 0, \lambda_\mathrm{C} \geq 0, \lambda_\mathrm{D} \geq 0.$$

Если все веса положительны, точка лежит строго внутри. Нулевой вес исключает соответствующую вершину из выпуклой комбинации. Поэтому после проверки неотрицательности число нулей определяет положение на границе:

Нулей	Положение	Геометрическое объяснение
0	Внутри	Положительны веса всех четырёх вершин
1	На грани	Комбинация трёх остальных вершин
2	На ребре	Комбинация двух остальных вершин
3	В вершине	Оставшийся вес равен 1

Четыре нуля невозможны, поскольку сумма весов равна 1. Если хотя бы один вес отрицателен, точка находится снаружи независимо от числа нулей. Например, веса (7/6; −1/6; 0; 0) задают точку на продолжении прямой AB, но вне ребра AB.

4.4.3 Ручное решение общей системы

Для выбранных вершин AB = (6; 0; 0), AC = (0; 6; 0), AD = (0; 0; 6), AP = (x; y; z). Расширенная матрица имеет вид:

$$(\mathrm{M} \mid \mathrm{AP}) = \begin{pmatrix} 6 & 0 & 0 & \mathrm{x} \\ 0 & 6 & 0 & \mathrm{y} \\ 0 & 0 & 6 & \mathrm{z} \end{pmatrix}.$$

Матрица уже треугольная. На прямом ходе занулять элементы не нужно. Разделим каждую строку на 6; коэффициенты при неизвестных образуют единичную матрицу:

$$(\mathrm{I} \mid \text{решение}) = \begin{pmatrix} 1 & 0 & 0 & \mathrm{x}/6 \\ 0 & 1 & 0 & \mathrm{y}/6 \\ 0 & 0 & 1 & \mathrm{z}/6 \end{pmatrix}.$$

$$\lambda_\mathrm{B} = \frac{\mathrm{x}}{6}, \quad \lambda_\mathrm{C} = \frac{\mathrm{y}}{6}, \quad \lambda_\mathrm{D} = \frac{\mathrm{z}}{6}, \quad \lambda_\mathrm{A} = \frac{6 - \mathrm{x} - \mathrm{y} - \mathrm{z}}{6}.$$

Для произвольных вершин матрица не обязана быть диагональной. Реализация из приложения А выполняет перестановки строк, исключение неизвестных и обратную подстановку; приведённые упрощённые формулы в коде не используются.

4.4.4 Ручные расчёты для тестовых точек

Точка P_1

$$\mathrm{P}_1(1;\ 1;\ 1): \quad 6\lambda_\mathrm{B} = 1; \quad 6\lambda_\mathrm{C} = 1; \quad 6\lambda_\mathrm{D} = 1.$$

$$\lambda_\mathrm{B} = 1/6; \quad \lambda_\mathrm{C} = 1/6; \quad \lambda_\mathrm{D} = 1/6.$$

$$\lambda_\mathrm{A} = 1 - 1/6 - 1/6 - 1/6 = 1/2.$$

Все четыре веса положительны, поэтому P_1 внутри. Проверка восстановления точки:

$$\mathrm{P}_1 = \mathrm{A}/2 + \mathrm{B}/6 + \mathrm{C}/6 + \mathrm{D}/6 = (1;\ 1;\ 1).$$

Точка P_2

$$\mathrm{P}_2(2;\ 2;\ 2): \quad 6\lambda_\mathrm{B} = 2; \quad 6\lambda_\mathrm{C} = 2; \quad 6\lambda_\mathrm{D} = 2.$$

$$\lambda_\mathrm{B} = 1/3; \quad \lambda_\mathrm{C} = 1/3; \quad \lambda_\mathrm{D} = 1/3.$$

$$\lambda_\mathrm{A} = 1 - 1/3 - 1/3 - 1/3 = 0.$$

Нулевой только вес A; P_2 лежит внутри грани BCD. Проверка восстановления точки:

$$\mathrm{P}_2 = \mathrm{B}/3 + \mathrm{C}/3 + \mathrm{D}/3 = (2;\ 2;\ 2).$$

Точка P_3

$$P_3(3;\ 3;\ 0):\quad 6\lambda_B = 3;\quad 6\lambda_C = 3;\quad 6\lambda_D = 0.$$
$$\lambda_B = 1/2;\quad \lambda_C = 1/2;\quad \lambda_D = 0.$$
$$\lambda_A = 1 - 1/2 - 1/2 - 0 = 0.$$

Веса A и D равны нулю; P_3 лежит внутри ребра BC. Проверка восстановления точки:

$$P_3 = B/2 + C/2 = (3;\ 3;\ 0).$$

Точка P_4

$$P_4(0;\ 0;\ 0):\quad 6\lambda_B = 0;\quad 6\lambda_C = 0;\quad 6\lambda_D = 0.$$
$$\lambda_B = 0;\quad \lambda_C = 0;\quad \lambda_D = 0.$$
$$\lambda_A = 1 - 0 - 0 - 0 = 1.$$

Три веса равны нулю; P_4 совпадает с вершиной A. Проверка восстановления точки:

$$P_4 = 1 \cdot A = (0;\ 0;\ 0).$$

Точка P_5

$$P_5(3;\ 3;\ 3):\quad 6\lambda_B = 3;\quad 6\lambda_C = 3;\quad 6\lambda_D = 3.$$
$$\lambda_B = 1/2;\quad \lambda_C = 1/2;\quad \lambda_D = 1/2.$$
$$\lambda_A = 1 - 1/2 - 1/2 - 1/2 = -1/2.$$

Вес A отрицателен; P_5 снаружи, за гранью BCD. Проверка восстановления точки:

$$P_5 = -A/2 + B/2 + C/2 + D/2 = (3;\ 3;\ 3).$$

Точка Q

$$Q(-1;\ 0;\ 0):\quad 6\lambda_B = -1;\quad 6\lambda_C = 0;\quad 6\lambda_D = 0.$$
$$\lambda_B = -1/6;\quad \lambda_C = 0;\quad \lambda_D = 0.$$
$$\lambda_A = 1 - (-1/6) - 0 - 0 = 7/6.$$

Вес B отрицателен; Q снаружи, несмотря на два нулевых веса. Проверка восстановления точки:

$$Q = 7A/6 - B/6 = (-1;\ 0;\ 0).$$

4.4.5 Сводка и программная реализация

Точка	λ_A	λ_B	λ_C	λ_D	Положение
P_1	1/2	1/6	1/6	1/6	Внутри
P_2	0	1/3	1/3	1/3	Грань BCD
P_3	0	1/2	1/2	0	Ребро BC
P_4	1	0	0	0	Вершина A
P_5	−1/2	1/2	1/2	1/2	Снаружи
Q	7/6	−1/6	0	0	Снаружи

В каждой строке сумма четырёх весов равна 1. Восстановление $\lambda_A A + \lambda_B B + \lambda_C C + \lambda_D D$ даёт координаты исходной точки. Эти две проверки позволяют контролировать результат решения системы.

Полный код метода 4 приведён в приложении A: solve3 решает систему, classify_barycentric вычисляет веса и положение, method_alternative возвращает положение для общей таблицы. Алгоритм:

1. Построить матрицу M из столбцов AB, AC, AD и правую часть AP.

2. Для каждого столбца выбрать максимальный по модулю ведущий элемент, переставить строки и исключить неизвестную из нижних строк. Нулевой ведущий элемент означает вырожденную систему.

3. Обратным ходом найти λ_B, λ_C и λ_D; затем вычислить $\lambda_A = 1 - \lambda_B - \lambda_C - \lambda_D$.

4. Если хотя бы один вес меньше $-\varepsilon$, вернуть «снаружи». Иначе подсчитать $|\lambda_i| \le \varepsilon$ и определить положение по таблице из пункта 4.4.2.

Принят $\varepsilon = 10^{-9}$. Это безразмерный допуск на веса, а не допуск на длину. Выбор главного элемента уменьшает ошибки исключения, но не устраняет чувствительность почти вырожденных тетраэдров. Сравнение ведущего элемента с нулём обнаруживает точное вырождение в выполняемой арифметике; оценка обусловленности в эту учебную реализацию не включена.

4.4.6 Проверка и вывод

На P_1–P_5 и Q программные веса совпали с таблицей, а положение - с ручным решением. Дополнительно проверены 80 случаев для двух тетраэдров и двух порядков вершин. Метод 4 не вызывает методы 1–3 и находит веса отдельным решением системы. При фиксированном размере 3 × 3 время и память на одну точку составляют O(1), на N точек - O(N) по времени.

Преимущество метода - непосредственное получение коэффициентов разложения точки по вершинам. Они пригодны для интерполяции цвета, температуры или других значений. Ограничение - необходимость решения системы и чувствительность к почти линейно зависимым столбцам M. Для многократной проверки точек в одном тетраэдре разложение матрицы можно подготовить заранее. Схема весов приведена в разделе 7.5.

5 Сравнение методов

5.1 Результаты на общих тестовых точках

В таблице сопоставлены ожидаемые положения и фактические результаты запуска четырёх методов. Во всех шести строках ручное решение и программные ответы совпали. Точка Q проверяет, что наличие нулей само по себе не означает принадлежность границе.

Точка	Ожидаемое положение	Метод 1	Метод 2	Метод 3	Метод 4
P_1	Внутри	Внутри	Внутри	Внутри	Внутри
P_2	Грань BCD	На грани	На грани	На грани	На грани
P_3	Ребро BC	На ребре	На ребре	На ребре	На ребре
P_4	Вершина A	В вершине	В вершине	В вершине	В вершине
P_5	Снаружи	Снаружи	Снаружи	Снаружи	Снаружи
Q	Снаружи	Снаружи	Снаружи	Снаружи	Снаружи

5.2 Преимущества и ограничения

Метод	Плюсы	Минусы
1 Объёмы	Наглядное сложение объёмов; одна функция для всех частей; различение грани, ребра и вершины.	Для каждой точки нужны четыре смешанных произведения. При вещественных координатах нужен допуск; почти вырожденные фигуры требуют повышенной точности.
2 Знаки	Проверка полупространств; видна нарушенная грань; нули различают положения на границе; отношения безразмерны.	В текущей реализации восемь смешанных произведений на точку; после подготовки s(T) - четыре. Почти вырожденные фигуры требуют повышенной точности.
3 Плоскости	Явные уравнения границ; коэффициенты можно подготовить для серии точек; отношения f(P)/f(T) не зависят от выбора масштаба и знака нормали.	Нужно составить четыре плоскости и сравнивать знаки с противоположными вершинами. Для почти вырожденных фигур требуется повышенная точность.
4 Веса вершин	Единая система уравнений; веса вершин пригодны для интерполяции; отрицательный вес указывает внешнюю точку.	Нужно решать систему. Почти вырожденная матрица чувствительна к округлению. При серии точек полезна предварительная факторизация.

5.3 Общий вывод о сравнении

Все четыре метода дали одинаковую классификацию на шести общих точках и 80 дополнительных случаях. Метод 1 использует сумму неотрицательных объёмов, метод 2 - знаки смешанных произведений, метод 3 - уравнения плоскостей, метод 4 - решение линейной системы. В точной арифметике критерии эквивалентны: $V_i/V_0 = |\lambda_i|$, $g_i = \lambda_i$, а отношения метода 2 совпадают с λ_i после перестановки порядка компонент. Метод 4 дополнительно даёт веса для интерполяции величин внутри элемента.

Численный допуск $\varepsilon = 10^{-9}$ применяется по-разному: метод 1 проверяет $|S - V_0| \le \varepsilon V_0$, методы 2–4 - каждое нормированное значение отдельно. Поэтому у самой границы ответы могут различаться. Для $P(1; 1; -4{,}5 \cdot 10^{-9})$ метод 1 возвращает «снаружи», а методы 2–4 - «на грани»; в точной арифметике точка внешняя. Это ожидаемое следствие разных допусков, а не противоречие точных критериев. Прикладная задача должна задавать, какую окрестность границы считать граничной.

5.4 Проверка полной программы

Кроме шести общих примеров выполнено 80 проверок с заранее известными весами. Использованы исходный тетраэдр и косой тетраэдр с вершинами A(1; 2; 3), B(5; 1; 0), C(2; 7; 1), D(0; 3; 8). Для каждого выполнена также перестановка B и C. Это проверяет независимость результата от ориентации и необходимость выбора ведущей строки в методе Гаусса.

Для каждого из четырёх наборов вершин построено 20 точек: 4 вершины, 6 середин рёбер, 4 центра граней, 1 внутренняя точка с весами 1/4; 4 внешние точки с одним весом −1/2 и тремя весами 1/2; 1 точка на продолжении ребра с весами (−1/4; 5/4; 0; 0). Ожидаемое положение определяется этими весами до запуска проверяемых функций. Всего: $4 \cdot 20 = 80$ случаев и 320 классификаций четырьмя методами.

Все ответы совпали с ожидаемыми. Для метода 4 дополнительно проверено восстановление исходной точки по найденным весам с ошибкой менее 10^{-12} на этих данных. Вырожденный тетраэдр отклонён всеми четырьмя методами; недопустимые значения ε (0, −1, 0,25 и NaN) вызывают ValueError. Переключатель объёмов проверен на всех шести точках. Эти проверки относятся к приведённым данным и не заменяют анализ численных ошибок на произвольных масштабах.

Измерения времени не проводились. Все четыре алгоритма имеют O(1) времени и памяти на точку. Для серии точек промежуточные величины, зависящие только от вершин, можно вычислить заранее.

6 Формулировка прикладной задачи

В компьютерной графике объёмную модель разбивают на тетраэдральные элементы. Для контрольной точки P требуется найти элемент с вершинами A, B, C, D, которому она принадлежит, и отличить внутреннее положение от попадания на границу.

Четыре метода решают эту задачу, а барицентрические веса позволяют интерполировать заданные в вершинах цвет или температуру. На общей границе точка может принадлежать нескольким элементам; правило выбора одного из них задаётся прикладной программой.

7 Визуализация результатов

7.1 Общий вид тетраэдра и тестовых точек

Визуализация выполнена в Python с помощью Matplotlib. На рисунке показаны общие вершины и все шесть тестовых точек. Их цвет и форма маркеров определены методом объёмов. По трём осям установлен одинаковый масштаб.

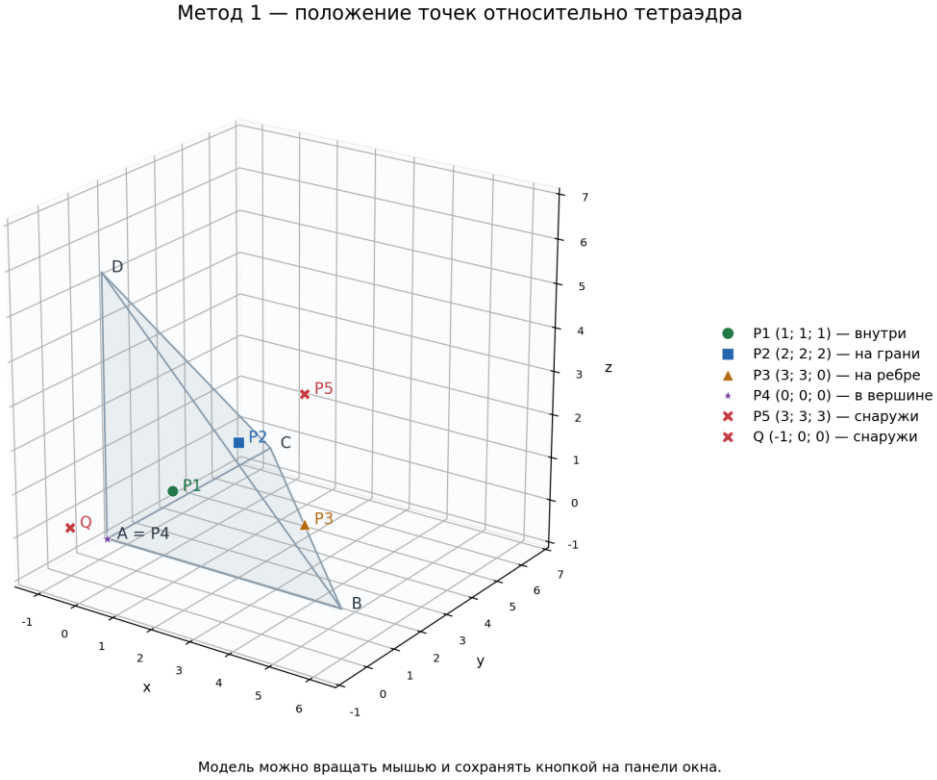


Рисунок 1. Тетраэдр ABCD и точки P_1–P_5 и Q в окне программы

P_1 находится внутри; P_2 - на грани BCD; P_3 - на ребре BC; P_4 совпадает с A. P_5 лежит снаружи за гранью BCD, а Q - на продолжении прямой AB за пределами ребра. В легенде программы обозначения P1, P2 и далее соответствуют P_1, P_2 и далее в формулах.

В графическом окне модель можно вращать мышью. На плоском изображении положение точки может восприниматься неоднозначно, поэтому классификация выполняется вычислениями; рисунок помогает проверить и объяснить результат.

7.2 Четыре объёма и выбор точки

Второе окно показывает тетраэдры PBCD, APCD, ABPD и ABCP. Серый каркас соответствует исходному ABCD. Цветной каркас и полупрозрачные грани показывают одну из четырёх частей. Слева расположен переключатель точек.

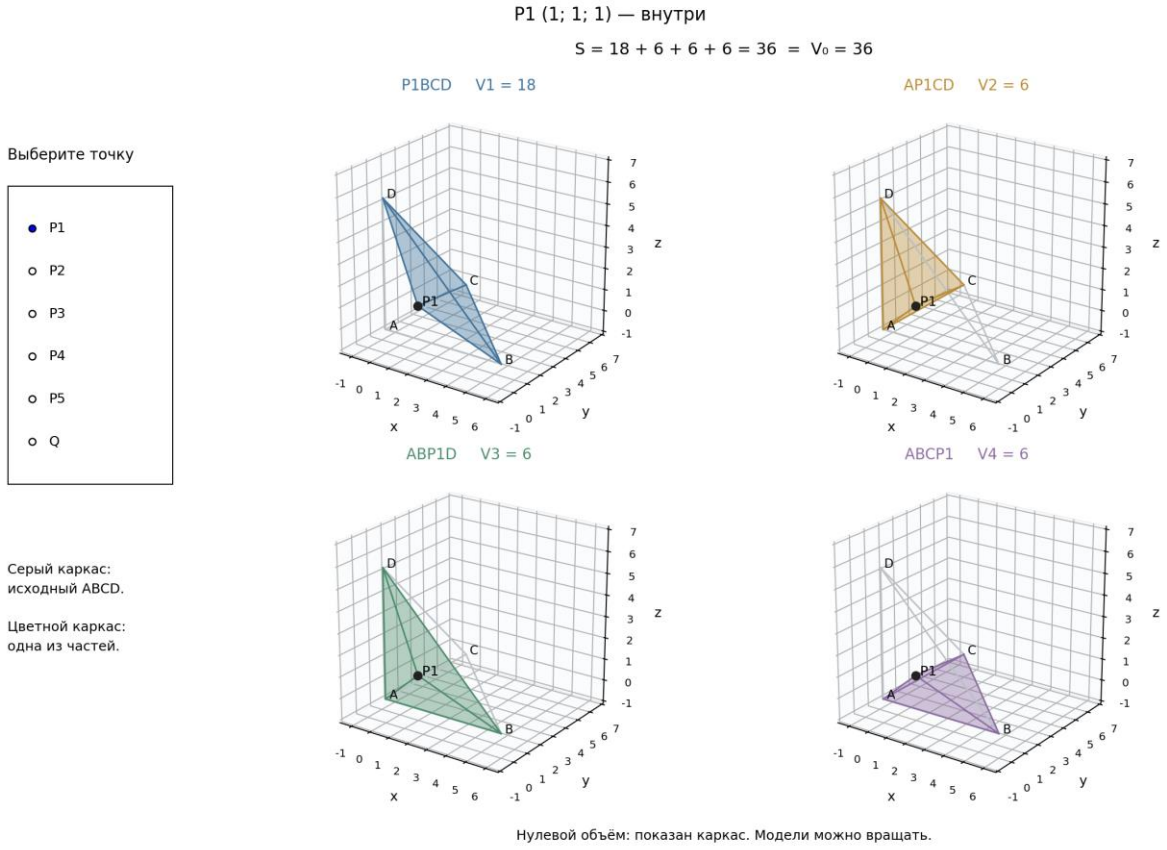


Рисунок 2. Интерактивное окно метода объёмов для точки P_1

$$S = V_1 + V_2 + V_3 + V_4 = 18 + 6 + 6 + 6 = 36 = V_0.$$

При выборе P_2 один объём становится нулевым, при выборе P_3 - два, при выборе P_4 - три. Для нулевых объёмов программа рисует только каркас. Для P_5 сумма равна 72, а для Q - 48; обе точки находятся снаружи.

Сумма и итоговое положение обновляются при переключении. Для внутренней точки цветные тетраэдры образуют разбиение ABCD. Для внешней точки их объёмы уже нельзя считать объёмами непересекающихся частей исходного тетраэдра.

Режим --no-plot --save-dir images сохраняет пять изображений: общий вид, объёмы, знаки, плоскости и барицентрические веса. Рисунки 1, 2, 4, 5 и 6 получены этим кодом. Геометрическая схема на рисунке 3 сохранена из материалов участника команды.

7.3 Схема метода знаков смешанных произведений

На рисунке 3 приведена геометрическая схема из материалов участника метода 2. Рисунок 4 строится кодом приложения А и показывает числовые результаты проверки всех четырёх граней для каждой точки.

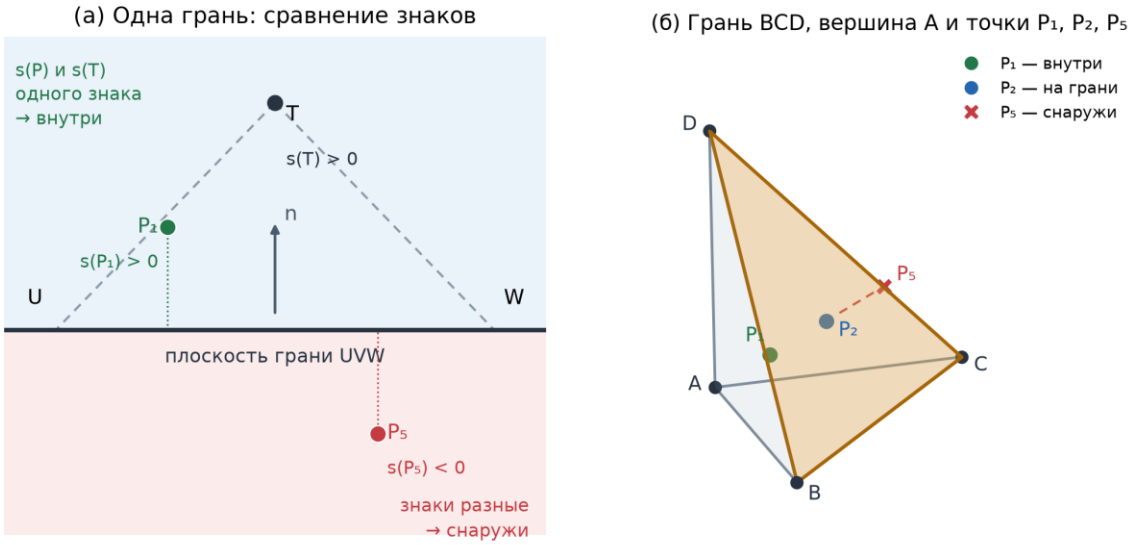


Рисунок 3. Схема метода знаков: а - одна грань UVW, противоположная вершина T и точки по разные стороны плоскости; б - грань BCD, вершина A, точки P_1, P_2, P_5 и проекция P_5 на грань

На панели (а) знак s(T) принят положительным. Точка P_1 лежит в том же полупространстве, что и T ($s(P_1) > 0$), поэтому для этой грани она проходит проверку; точка P_5 лежит в другом полупространстве ($s(P_5) < 0$), и для неё знаки разные. На панели (б) грань BCD выделена, P_2 лежит на ней, а P_5 - на продолжении нормали к грани за P_2. Поэтому $s_4(P_5)$ имеет знак, противоположный $s_4(A)$, и точка P_5 находится снаружи. P_1 расположена внутри тетраэдра.

Пометка «внутри» на панели (а) означает внутреннюю сторону одной плоскости. Для принадлежности всему тетраэдру нужно пройти проверки всех четырёх граней.

7.3.1 Значения отношений для тестовых точек

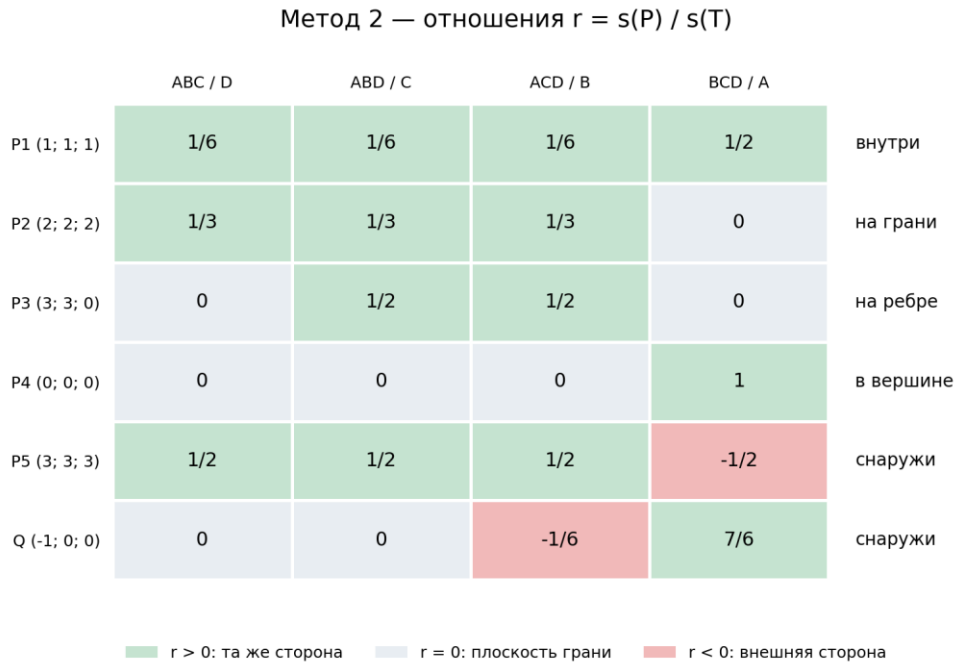


Рисунок 4. Нормированные смешанные произведения $r_i = s_i(P)/s_i(T)$ для тестовых точек: зелёный - $r > 0$, красный - $r < 0$, светлый - $r = 0$

На рисунке 4 каждая строка соответствует точке, а столбцы - граням ABC, ABD, ACD и BCD. У точки P_2 ровно один нуль (грань BCD), у P_3 - два нуля (грани ABC и BCD, ребро BC), у P_4 - три нуля (вершина A). У P_5 одно отношение отрицательно, поэтому точка снаружи. Строка Q показывает, что два нуля сами по себе не означают ребро: отношение r_3 отрицательно, и точка находится снаружи за гранью ACD.

7.4 Схема метода уравнений плоскостей

Схема построена кодом приложения А с помощью Matplotlib. Каждая панель соответствует одной грани: синим выделен треугольник, бледным кругом - фрагмент её плоскости, ромбом - противоположная вершина. Использованы общие вершины и тестовые точки.

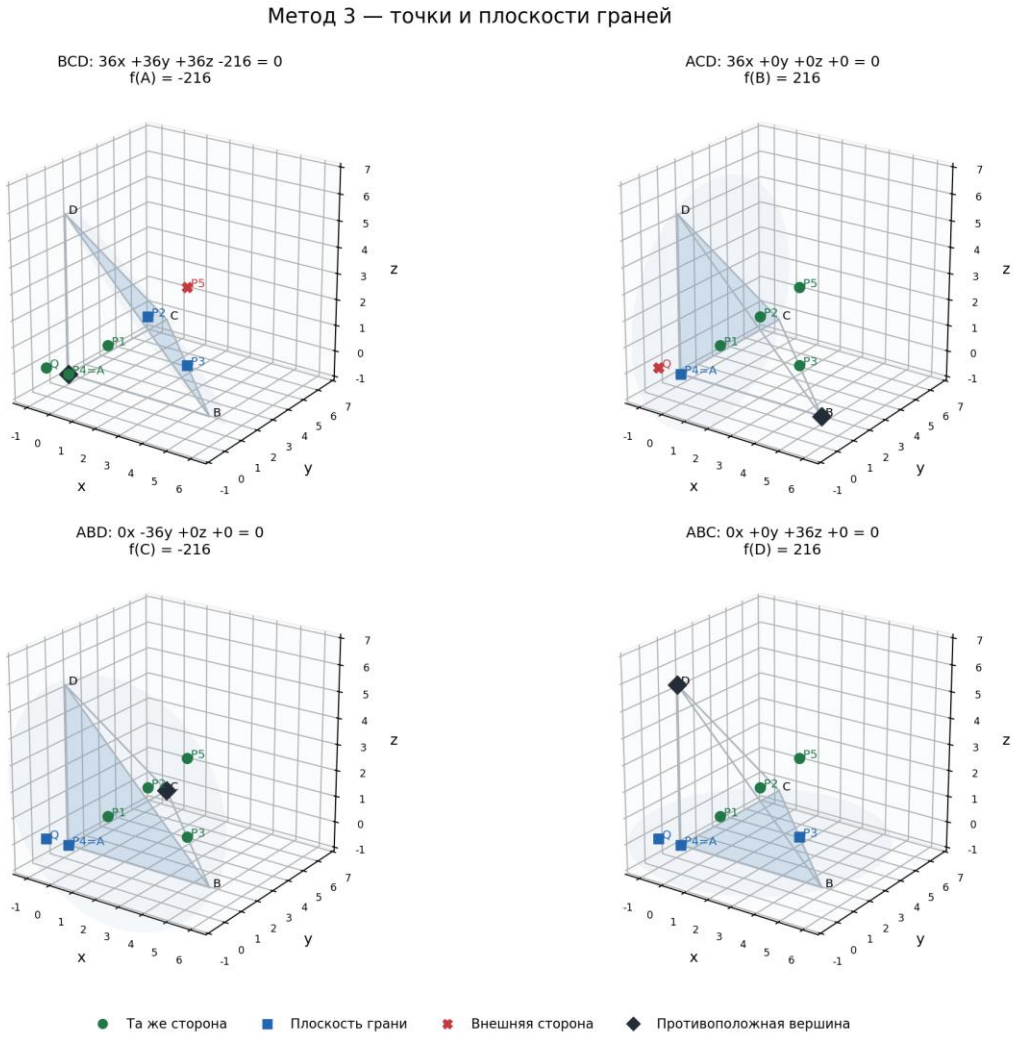


Рисунок 5. Плоскости граней и положение точек P_1–P_5 и Q относительно каждой из них

Цвет и форма маркера показывают знак f(P) относительно f(t): зелёный круг - тот же знак (точка с той же стороны, что и t), синий квадрат - f(P) = 0 (точка в плоскости грани), красный крест - знак противоположный (точка за гранью). В заголовке каждой панели указаны уравнение плоскости и значение f(t).

Точка P_1 на всех панелях зелёная - она внутри. P_2 синяя на панели BCD, P_3 - на панелях BCD и ABC, P_4 (вершина A) - на панелях ACD, ABD и ABC, то есть число синих маркеров равно числу нулевых g_i. Красные кресты показывают внешние точки: P_5 - за гранью BCD, Q - за гранью ACD. Для внешней точки достаточно одной красной панели.

7.5 Схема метода барицентрических координат

На каждом графике показаны четыре веса λ_A, λ_B, λ_C и λ_D, полученные решением системы методом Гаусса. Синий столбец соответствует неотрицательному весу, красный - отрицательному. Нулевой вес подписан на горизонтальной оси. Все значения совпадают с ручной таблицей раздела 4.4.

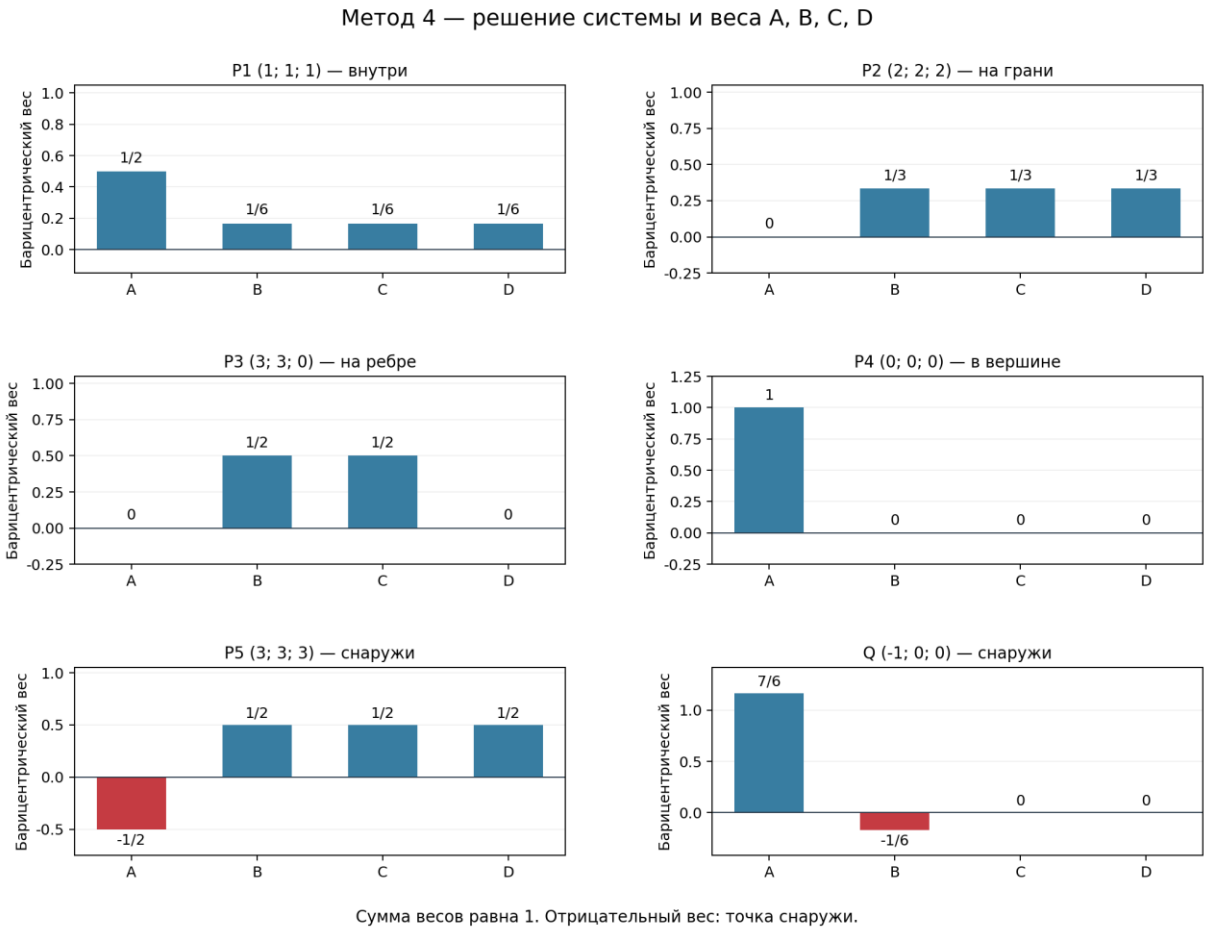


Рисунок 6. Барицентрические веса вершин A, B, C, D для точек P_1–P_5 и Q

У P_1 четыре положительных веса; у P_2 нулевой вес A; у P_3 нулевые веса A и D; у P_4 единственный ненулевой вес равен 1 и относится к A. У P_5 отрицателен вес A, у Q - вес B. Поэтому последние две точки внешние, хотя Q имеет ещё и два нулевых веса.

Высота столбца - коэффициент при вершине в выражении $P = \lambda_A A + \lambda_B B + \lambda_C C + \lambda_D D$. Сумма весов на каждой панели равна 1. Рисунок построен Matplotlib функцией create_barycentric_plot из приложения А и встроен в документ как изображение.

Приложение А Полный исходный код программы

Ниже приведён весь код project.py: данные, четыре метода, вывод результатов, пять рисунков, обработка параметров запуска и main. Части листинга следует объединить в указанном порядке. Вычисления используют стандартную библиотеку Python; для графики нужна Matplotlib. Комментарии объясняют основные действия.

А1 Импорты и исходные данные

```
"""Четыре метода проверки принадлежности точки тетраэдру.

Код полностью приведён в приложении А к отчёту.
Сохраните его как project.py, сохранив отступы.
Расчёты: python project.py --no-plot
Рисунки: python project.py
Своя точка: python project.py --point 1 2 1
Сохранение PNG: python project.py --no-plot --save-dir images
Для рисунков: python -m pip install matplotlib

Ввод через input() не требуется: общие данные находятся ниже.
Метод 4 независимо решает систему методом Гаусса.
В точной арифметике все четыре критерия эквивалентны.
У самой границы правила допуска метода 1 и методов 2–4 различаются.
"""

import argparse
from itertools import combinations
from math import isfinite
from pathlib import Path


# ---------------------- Исходные данные ----------------------
# Порядок важен: первая вершина - A, затем B, C, D.
VERTICES = ((0, 0, 0), (6, 0, 0), (0, 6, 0), (0, 0, 6))

# Можно добавить строку ("P6", (x, y, z)) или изменить координаты.
# Названия точек должны быть уникальными. Имя "P" оставлено для --point.
TEST_POINTS = [
    ("P1", (1, 1, 1)),      # Внутри.
    ("P2", (2, 2, 2)),      # На грани BCD.
    ("P3", (3, 3, 0)),      # На ребре BC.
    ("P4", (0, 0, 0)),      # В вершине A.
    ("P5", (3, 3, 3)),      # Снаружи.
    ("Q", (-1, 0, 0)),      # Снаружи, хотя два объёма равны нулю.
]

POSITION_STYLES = {
    "внутри": ("#227849", "o"),
    "на грани": ("#2467af", "s"),
    "на ребре": ("#b66b10", "^"),
    "в вершине": ("#7645a7", "*"),
    "снаружи": ("#c53b42", "X"),
}
```

A2 Метод объёмов

```
# -------------------- Метод 1 и векторы ----------------------
def subtract(a, b):
    """Вектор a - b."""
    return tuple(a[i] - b[i] for i in range(3))


def mixed(u, v, w):
    """Смешанное произведение u · (v × w)."""
    return (
        u[0] * (v[1] * w[2] - v[2] * w[1])
        - u[1] * (v[0] * w[2] - v[2] * w[0])
        + u[2] * (v[0] * w[1] - v[1] * w[0])
    )


def volume(a, b, c, d):
    """Неотрицательный объём тетраэдра с вершинами a, b, c, d."""
    ab = subtract(b, a)
    ac = subtract(c, a)
    ad = subtract(d, a)
    return abs(mixed(ab, ac, ad)) / 6


def classify_point(a, b, c, d, p, eps=1e-9):
    """Возвращает положение, исходный объём и четыре объёма частей.

    eps задаёт относительный допуск по объёму.
    Предполагаются конечные координаты и невырожденный тетраэдр.
    """
    if not 0 < eps < 0.25:
        raise ValueError("Требуется 0 < eps < 0.25")
    total = volume(a, b, c, d)
    if total == 0:
        raise ValueError("Тетраэдр вырожден: его объём равен нулю")

    parts = (
        volume(p, b, c, d),
        volume(a, p, c, d),
        volume(a, b, p, d),
        volume(a, b, c, p),
    )
    tolerance = eps * total

    # Сначала проверяем принадлежность, затем вид границы.
    if abs(sum(parts) - total) > tolerance:
        return "снаружи", total, parts

    zeros = sum(part <= tolerance for part in parts)
    positions = {0: "внутри", 1: "на грани",
                 2: "на ребре", 3: "в вершине"}
    return positions[zeros], total, parts


def method_volumes(a, b, c, d, p, eps=1e-9):
    """Обёртка: для общей таблицы нужен только текст положения."""
    position, total, parts = classify_point(a, b, c, d, p, eps)
    return position
```

А3 Знаки смешанных произведений

```
# ---------------------- Метод 2 ----------------------
def classify_signs(a, b, c, d, p, eps=1e-9):
    """Положение по знакам смешанных произведений.

    Порядок граней: ABC, ABD, ACD, BCD.
    Возвращает положение, s_P, s_T и отношения s_P / s_T.
    """
    if not 0 < eps < 0.25:
        raise ValueError("Требуется 0 < eps < 0.25")

    faces = (
        (a, b, c, d),
        (a, b, d, c),
        (a, c, d, b),
        (b, c, d, a),
    )
    products, opposite, ratios = [], [], []
    for u, v, w, t in faces:
        uv = subtract(v, u)
        uw = subtract(w, u)
        s_p = mixed(uv, uw, subtract(p, u))
        s_t = mixed(uv, uw, subtract(t, u))
        if s_t == 0:
            raise ValueError("Тетраэдр вырожден")
        products.append(s_p)
        opposite.append(s_t)
        ratios.append(s_p / s_t)

    if any(r < -eps for r in ratios):
        return "снаружи", products, opposite, ratios

    zeros = sum(abs(r) <= eps for r in ratios)
    positions = {0: "внутри", 1: "на грани",
                 2: "на ребре", 3: "в вершине"}
    return positions[zeros], products, opposite, ratios


def method_signs(a, b, c, d, p, eps=1e-9):
    """Обёртка для общего сравнения методов."""
    position, *_ = classify_signs(a, b, c, d, p, eps)
    return position
```

А4 Уравнения плоскостей

```
# ---------------------- Метод 3 ----------------------
def cross(u, v):
    """Векторное произведение u × v."""
    return (
        u[1] * v[2] - u[2] * v[1],
        u[2] * v[0] - u[0] * v[2],
        u[0] * v[1] - u[1] * v[0],
    )


def plane_through(u, v, w):
    """Плоскость через u, v, w: f(x, y, z) = nx*x + ny*y + nz*z + k.

    Возвращает коэффициенты (nx, ny, nz, k).
    Нормаль n = (v - u) × (w - u), k = -(n · u).
    """
```

```
    n = cross(subtract(v, u), subtract(w, u))
    k = -(n[0] * u[0] + n[1] * u[1] + n[2] * u[2])
    return n[0], n[1], n[2], k


def plane_value(plane, p):
    """Значение f(P) = nx*px + ny*py + nz*pz + k."""
    nx, ny, nz, k = plane
    return nx * p[0] + ny * p[1] + nz * p[2] + k


def classify_planes(a, b, c, d, p, eps=1e-9):
    """Метод 3: возвращает положение и четыре отношения g = f(P) / f(t).

    Грани и противоположные вершины t: BCD/A, ACD/B, ABD/C, ABC/D.
    Порядок совпадает с объёмами метода 1: g_i = ±V_i / V_0 (PBCD, APCD,
    ABPD, ABCP).
    g >= 0 означает, что P не лежит по другую сторону от t относительно
    плоскости грани; g = 0 означает, что P лежит в этой плоскости.
    Отношение не зависит от масштаба и знака нормали, поэтому допуск eps
    безразмерный.
    """
    if not 0 < eps < 0.25:
        raise ValueError("Требуется 0 < eps < 0.25")
    ratios = []
    for u, v, w, t in ((b, c, d, a), (a, c, d, b), (a, b, d, c), (a, b, c, d)):
        plane = plane_through(u, v, w)
        f_t = plane_value(plane, t)
        if f_t == 0:
            raise ValueError("Тетраэдр вырожден: вершина лежит в плоскости грани")
        ratios.append(plane_value(plane, p) / f_t)

    # Разные знаки f(P) и f(t) хотя бы для одной грани: точка снаружи.
    if any(g < -eps for g in ratios):
        return "снаружи", ratios

    zeros = sum(abs(g) <= eps for g in ratios)
    positions = {0: "внутри", 1: "на грани",
                 2: "на ребре", 3: "в вершине"}
    return positions[zeros], ratios


def method_planes(a, b, c, d, p, eps=1e-9):
    """МЕТОД 3: уравнения плоскостей (обёртка: только текст положения)."""
    position, ratios = classify_planes(a, b, c, d, p, eps)
    return position
```

А5 Барицентрические координаты и метод Гаусса

```
# ---------------------- Метод 4 ----------------------
def solve3(matrix, rhs):
    """Система 3 x 3: метод Гаусса с выбором главного элемента."""
    aug = [[float(x) for x in row] + [float(value)]
           for row, value in zip(matrix, rhs)]
    for k in range(3):
        # Выбираем наибольший по модулю элемент текущего столбца.
        pivot = max(range(k, 3), key=lambda i: abs(aug[i][k]))
        if aug[pivot][k] == 0:
            raise ValueError("Тетраэдр вырожден: система не имеет ранга 3")
        aug[k], aug[pivot] = aug[pivot], aug[k]
        for i in range(k + 1, 3):
            factor = aug[i][k] / aug[k][k]
```

```
            aug[i][k] = 0.0
            for j in range(k + 1, 4):
                aug[i][j] -= factor * aug[k][j]
    # Обратный ход: сначала третья неизвестная, затем вторая и первая.
    answer = [0.0] * 3
    for i in range(2, -1, -1):
        rest = sum(aug[i][j] * answer[j] for j in range(i + 1, 3))
        answer[i] = (aug[i][3] - rest) / aug[i][i]
    return answer


def classify_barycentric(a, b, c, d, p, eps=1e-9):
    """Положение и веса A, B, C, D, найденные решением системы."""
    if not 0 < eps < 0.25:
        raise ValueError("Требуется 0 < eps < 0.25")
    # Столбцы матрицы: AB, AC, AD; правая часть: AP.
    matrix = [[b[i] - a[i], c[i] - a[i], d[i] - a[i]]
              for i in range(3)]
    tail = solve3(matrix, subtract(p, a))
    weights = [1 - sum(tail)] + tail
    if any(value < -eps for value in weights):
        return "снаружи", weights
    zeros = sum(abs(value) <= eps for value in weights)
    positions = {0: "внутри", 1: "на грани",
                 2: "на ребре", 3: "в вершине"}
    return positions[zeros], weights


def method_alternative(a, b, c, d, p, eps=1e-9):
    """Метод 4: барицентрические координаты через систему уравнений."""
    position, weights = classify_barycentric(a, b, c, d, p, eps)
    return position
```

A6 Общий вывод результатов

```
# Здесь указаны функции, которые программа вызывает при сравнении.
# ВНИМАНИЕ: пишем имя функции БЕЗ скобок, чтобы сохранить саму функцию.
METHODS = [
    ("1. Объёмы", method_volumes),
    ("2. Знаки смешанных произведений", method_signs),
    ("3. Уравнения плоскостей", method_planes),
    ("4. Барицентрические координаты", method_alternative),
]


def coordinates(point):
    """Короткая запись координат для консоли и подписей."""
    return "(" + "; ".join(f"{value:g}" for value in point) + ")"


def print_results(vertices, points):
    """Подробные расчёты метода 1 и сравнение подключённых методов."""
    print("\nМЕТОД 1. Порядок частей: PBCD, APCD, ABPD, ABCP")
    for name, p in points:
        position, total, parts = classify_point(*vertices, p)
        values = ", ".join(f"{value:g}" for value in parts)
        print(f"{name} {coordinates(p)}: {position}")
        print(f"  V = {total:g}; части = ({values}); "
              f"сумма = {sum(parts):g}")

    print("\nМЕТОД 2. Порядок граней: ABC, ABD, ACD, BCD")
```

```
    for name, p in points:
        position, products, opposite, ratios = classify_signs(*vertices, p)
        print(f"{name} {coordinates(p)}: {position}")
        print(f"  s = {coordinates(products)}; s* = {coordinates(opposite)}")
        print(f"  r = {coordinates(ratios)}")
    print("\nМЕТОД 3. Порядок граней: BCD, ACD, ABD, ABC")
    for name, p in points:
        position, ratios = classify_planes(*vertices, p)
        print(f"{name} {coordinates(p)}: {position}; g = {coordinates(ratios)}")

    print("\nМЕТОД 4. Порядок весов: A, B, C, D")
    for name, p in points:
        position, weights = classify_barycentric(*vertices, p)
        print(f"{name} {coordinates(p)}: {position}")
        print(f"  lambda = {coordinates(weights)}; сумма = {sum(weights):g}")

    print("\nСРАВНЕНИЕ МЕТОДОВ")
    for title, method in METHODS:
        print(title)
        for name, p in points:
            print(f"  {name}: {method(*vertices, p)}")
```

А7 Построение тетраэдра и настройка осей

```
# ------------------------ Рисунки ----------------------------
def draw_tetrahedron(ax, vertices, color, alpha=0.0):
    """Шесть рёбер и четыре полупрозрачные треугольные грани."""
    from mpl_toolkits.mplot3d.art3d import Poly3DCollection

    for i, j in combinations(range(4), 2):
        # zip превращает две точки в пары координат x, y, z.
        ax.plot(*zip(vertices[i], vertices[j]), color=color, linewidth=1.2)
    # Нулевая часть может стать треугольником, отрезком или точкой.
    # Для неё показываем только каркас, не изображая ложный объём.
    if alpha > 0 and volume(*vertices) > 0:
        faces = [[vertices[i] for i in ids]
                 for ids in combinations(range(4), 3)]
        ax.add_collection3d(Poly3DCollection(
            faces, facecolors=color, alpha=alpha, edgecolors="none"))


def setup_axes(ax, vertices, points):
    """Одинаковый масштаб по x, y, z; все точки помещаются в кадр."""
    all_points = list(vertices) + [p for name, p in points]
    lows = [min(p[i] for p in all_points) for i in range(3)]
    highs = [max(p[i] for p in all_points) for i in range(3)]
    span = max(highs[i] - lows[i] for i in range(3)) * 1.18
    span = max(span, 1e-12)
    limits = []
    for low, high in zip(lows, highs):
        center = (low + high) / 2
        limits.append((center - span / 2, center + span / 2))
    ax.set(xlim=limits[0], ylim=limits[1], zlim=limits[2],
           xlabel="x", ylabel="y", zlabel="z")
    ax.set_box_aspect((1, 1, 1))
    ax.view_init(elev=22, azim=-55)
    ax.tick_params(labelsize=8)
    for axis in (ax.xaxis, ax.yaxis, ax.zaxis):
        axis.pane.set_facecolor((0.97, 0.98, 0.99, 1))
```

A8 Общий вид и интерактивное окно объёмов

```
def create_plots(vertices, points, selected_name):
    """Создаёт общий рисунок и четыре части для выбранной точки."""
    import matplotlib.pyplot as plt
    from matplotlib.lines import Line2D
    from matplotlib.widgets import RadioButtons

    plt.rcParams["font.family"] = "DejaVu Sans"
    plt.rcParams["axes.unicode_minus"] = False
    point_map = dict(points)

    # Окно 1: все точки и их положения, рассчитанные методом объёмов.
    overview = plt.figure(figsize=(12.5, 8))
    overview.canvas.manager.set_window_title("Тетраэдр и тестовые точки")
    ax = overview.add_subplot(111, projection="3d", computed_zorder=False)
    setup_axes(ax, vertices, points)
    draw_tetrahedron(ax, vertices, "#879bac", alpha=0.07)
    handles = []
    for name, p in points:
        status, total, parts = classify_point(*vertices, p)
        color, marker = POSITION_STYLES[status]
        ax.scatter(*p, color=color, marker=marker, s=65, depthshade=False,
                   edgecolors="white", linewidths=0.5, zorder=6)
        if p not in vertices:
            ax.text(*p, "  " + name, color=color, fontsize=11, zorder=7)
        handles.append(Line2D(
            [0], [0], color="none", marker=marker, markerfacecolor=color,
            markeredgecolor="white", markersize=9,
            label=f"{name} {coordinates(p)} - {status}"))
    for name, vertex in zip("ABCD", vertices):
        coinciding = [label for label, p in points if p == vertex]
        label = " = ".join([name] + coinciding)
        ax.text(*vertex, "  " + label, fontsize=11, color="#273341")
    overview.suptitle("Метод 1 - положение точек относительно тетраэдра",
                      fontsize=14)
    overview.legend(handles=handles, loc="center left", frameon=False,
                    bbox_to_anchor=(0.69, 0.51), fontsize=10)
    overview.text(0.50, 0.035,
                  "Модель можно вращать мышью и сохранять кнопкой на панели окна.",
                  ha="center", fontsize=10)
    overview.subplots_adjust(left=0.01, right=0.70, bottom=0.09, top=0.90)

    # Окно 2: переключатель выбирает точку для четырёх объёмов.
    parts_figure = plt.figure(figsize=(13, 9))
    parts_figure.canvas.manager.set_window_title("Четыре объёма для выбранной точки")
    axes = [parts_figure.add_subplot(2, 2, i + 1, projection="3d",
                                     computed_zorder=False) for i in range(4)]
    parts_figure.subplots_adjust(left=0.20, right=0.97, top=0.87,
                                 bottom=0.08, wspace=0.04, hspace=0.14)
    heading = parts_figure.suptitle("", fontsize=13, y=0.975)
    summary = parts_figure.text(0.59, 0.92, "", ha="center", fontsize=12)
    parts_figure.text(0.025, 0.80, "Выберите точку", fontsize=11)
    selector_axes = parts_figure.add_axes([0.025, 0.43, 0.13, 0.34])
    names = [name for name, p in points]
    selector = RadioButtons(selector_axes, names, active=names.index(selected_name))
    parts_figure.text(0.025, 0.34,
                      "Серый каркас:\nисходный ABCD.\n\n"
                      "Цветной каркас:\nодна из частей.",
                      va="top", fontsize=10, linespacing=1.6)
    parts_figure.text(0.59, 0.025,
                      "Нулевой объём: показан каркас. Модели можно вращать.",
                      ha="center", fontsize=10)
```

```
    def update(selected):
        # Обработчик вызывается при выборе точки слева.
        p = point_map[selected]
        status, total, parts = classify_point(*vertices, p)
        heading.set_text(f"{selected} {coordinates(p)} - {status}")
        relation = "=" if abs(sum(parts) - total) <= 1e-9 * total else ">"
        summary.set_text(f"S = {' + '.join(f'{v:g}' for v in parts)}"
                         f" = {sum(parts):g}  {relation}  V₀ = {total:g}")
        part_names = ["PBCD", "APCD", "ABPD", "ABCP"]
        colors = ["#3e719b", "#bb8b35", "#4c8c6b", "#8d6ca4"]
        for i, (axis, color) in enumerate(zip(axes, colors)):
            # Сохраняем выбранный пользователем ракурс при смене точки.
            elevation, azimuth = axis.elev, axis.azim
            axis.clear()
            setup_axes(axis, vertices, points)
            if getattr(axis, "_has_point", False):
                axis.view_init(elev=elevation, azim=azimuth)
            axis._has_point = True
            draw_tetrahedron(axis, vertices, "#bbc1c7")
            sub_vertices = list(vertices)
            sub_vertices[i] = p
            fill = 0.22 if parts[i] > 1e-9 * total else 0.0
            draw_tetrahedron(axis, sub_vertices, color, alpha=fill)
            for label, vertex in zip("ABCD", vertices):
                if vertex != p:
                    axis.text(*vertex, " " + label, fontsize=9)
            axis.scatter(*p, color="#222222", s=35, depthshade=False, zorder=6)
            matching = [label for label, vertex in zip("ABCD", vertices)
                        if vertex == p]
            point_label = " = ".join([selected] + matching)
            axis.text(*p, " " + point_label, fontsize=10, zorder=7)
            title = part_names[i].replace("P", selected)
            axis.set_title(f"{title}     V{i + 1} = {parts[i]:g}",
                           color=color, fontsize=11)
        parts_figure.canvas.draw_idle()

    selector.on_clicked(update)
    update(selected_name)
    # Ссылка на selector должна жить до закрытия окон, иначе кнопки
    # могут перестать реагировать после очистки объекта из памяти.
    return overview, parts_figure, selector
```

A9 Визуализация знаков

```
def create_signs_plot(vertices, points):
    """Метод 2: таблица отношений по граням ABC, ABD, ACD, BCD."""
    import matplotlib.pyplot as plt
    from matplotlib.patches import Rectangle, Patch
    from fractions import Fraction

    fig, ax = plt.subplots(figsize=(11, max(5, 0.65 * len(points) + 2)))
    fig.canvas.manager.set_window_title("Метод 2 - знаки смешанных произведений")
    for row, (name, p) in enumerate(points):
        position, products, opposite, ratios = classify_signs(*vertices, p)
        for col, value in enumerate(ratios):
            color = "#e8edf2" if abs(value) <= 1e-9 else (
                "#f1b9b9" if value < 0 else "#c5e3d0")
            ax.add_patch(Rectangle((col, row), 1, 1, facecolor=color,
                                   edgecolor="white", linewidth=2))
            fraction = Fraction(float(value)).limit_denominator(1000)
```

```
            label = (str(fraction) if abs(float(fraction) - value) < 1e-10
                     else f"{value:.4g}")
            ax.text(col + .5, row + .5, label, ha="center", va="center", fontsize=12)
        ax.text(4.15, row + .5, position, va="center", fontsize=11)
    ax.set_xlim(0, 5.6)
    ax.set_ylim(len(points), 0)
    ax.set_xticks([.5, 1.5, 2.5, 3.5], ["ABC / D", "ABD / C", "ACD / B", "BCD / A"])
    ax.xaxis.tick_top()
    ax.set_yticks([i + .5 for i in range(len(points))],
                  [f"{name} {coordinates(p)}" for name, p in points])
    ax.tick_params(length=0, pad=8)
    for spine in ax.spines.values():
        spine.set_visible(False)
    fig.suptitle("Метод 2 - отношения r = s(P) / s(T)", fontsize=15)
    handles = [Patch(facecolor=c, label=t) for c, t in [
        ("#c5e3d0", "r > 0: та же сторона"),
        ("#e8edf2", "r = 0: плоскость грани"),
        ("#f1b9b9", "r < 0: внешняя сторона")]]
    fig.legend(handles=handles, loc="lower center", ncol=3, frameon=False)
    fig.subplots_adjust(left=.19, right=.98, top=.84, bottom=.14)
    return fig
```

A10 Визуализация плоскостей

```
def create_planes_plot(vertices, points):
    """Метод 3: четыре плоскости и знаки f(P)/f(T) для всех точек."""
    import matplotlib.pyplot as plt
    from matplotlib.lines import Line2D
    from mpl_toolkits.mplot3d.art3d import Poly3DCollection
    from math import sqrt, sin, cos, pi

    a, b, c, d = vertices
    faces = [(b, c, d, a, "BCD", "A"), (a, c, d, b, "ACD", "B"),
             (a, b, d, c, "ABD", "C"), (a, b, c, d, "ABC", "D")]
    fig = plt.figure(figsize=(12, 10))
    fig.canvas.manager.set_window_title("Метод 3 - уравнения плоскостей")
    extent = max(sqrt(sum((v[j]-w[j])**2 for j in range(3)))
                 for v, w in combinations(vertices, 2))
    for i, (u, v, w, t, face_name, opposite_name) in enumerate(faces):
        ax = fig.add_subplot(2, 2, i + 1, projection="3d", computed_zorder=False)
        setup_axes(ax, vertices, points)
        draw_tetrahedron(ax, vertices, "#acb6bf")
        plane = plane_through(u, v, w)
        f_t = plane_value(plane, t)
        center = tuple((u[j]+v[j]+w[j])/3 for j in range(3))
        # Два единичных вектора в плоскости задают видимый круглый фрагмент.
        e = subtract(v, u)
        norm_e = sqrt(sum(x*x for x in e))
        e = tuple(x/norm_e for x in e)
        q = cross(plane[:3], e)
        norm_q = sqrt(sum(x*x for x in q))
        q = tuple(x/norm_q for x in q)
        disk = [tuple(center[j] + .55*extent*(cos(k*pi/30)*e[j] +
                      sin(k*pi/30)*q[j]) for j in range(3)) for k in range(60)]
        ax.add_collection3d(Poly3DCollection([disk], facecolors="#98b4d2", alpha=.09))
        ax.add_collection3d(Poly3DCollection(
            [[u,v,w]], facecolors="#6d98c5", alpha=.24))
        # Ромб показывает противоположную вершину, с которой сравнивают знак.
        ax.scatter(*t, marker="D", s=75, color="#252e38", depthshade=False, zorder=4)
        for name, p in points:
            g = plane_value(plane, p)/f_t
```

```
            color, marker = ("#2467af", "s") if abs(g) <= 1e-9 else (
                ("#c53b42", "X") if g < 0 else ("#227849", "o"))
            ax.scatter(*p, s=45, color=color, marker=marker,
                       depthshade=False, zorder=6)
            aliases = [label for label, vertex in zip("ABCD", vertices)
                       if vertex == p]
            ax.text(*p, " " + "=".join([name] + aliases),
                    fontsize=8, color=color, zorder=7)
        for label, vertex in zip("ABCD", vertices):
            if not any(p == vertex for _, p in points):
                ax.text(*vertex, " " + label, fontsize=8)
        nx, ny, nz, k = plane
        equation = f"{nx:g}x {ny:+g}y {nz:+g}z {k:+g} = 0"
        ax.set_title(f"{face_name}: {equation}\nf({opposite_name}) = {f_t:g}",
                     fontsize=10)
        ax.tick_params(labelsize=7)
    fig.suptitle("Метод 3 - точки и плоскости граней", fontsize=14)
    handles = [Line2D([], [], linestyle="none", marker=m, color=c, label=l)
               for m,c,l in [("o","#227849","Та же сторона"),
                             ("s","#2467af","Плоскость грани"),
                             ("X","#c53b42","Внешняя сторона"),
                             ("D","#252e38","Противоположная вершина")]]
    fig.legend(handles=handles, loc="lower center", ncol=4, frameon=False, fontsize=9)
    fig.subplots_adjust(left=.04, right=.97, top=.90, bottom=.08,
                        wspace=.08, hspace=.19)
    return fig
```

A11 Визуализация барицентрических координат

```
def create_barycentric_plot(vertices, points):
    """Метод 4: веса четырёх вершин для каждой тестовой точки."""
    import matplotlib.pyplot as plt
    from fractions import Fraction

    rows = (len(points) + 1) // 2
    fig, axes = plt.subplots(rows, 2, figsize=(12, 3 * rows),
                             squeeze=False)
    fig.canvas.manager.set_window_title("Метод 4 - веса вершин")
    for ax, (name, p) in zip(axes.flat, points):
        status, weights = classify_barycentric(*vertices, p)
        colors = ["#c53b42" if x < -1e-9 else "#387da1"
                  for x in weights]
        ax.bar(list("ABCD"), weights, color=colors, width=.55)
        ax.axhline(0, color="#263747", linewidth=.8)
        for i, value in enumerate(weights):
            f = Fraction(value).limit_denominator(1000)
            label = str(f) if abs(float(f) - value) < 1e-10 else f"{value:.4g}"
            ax.text(i, value + (.04 if value >= 0 else -.04), label,
                    ha="center", va="bottom" if value >= 0 else "top")
        ax.set_ylim(min(-.15, min(weights) - .25), max(1.05, max(weights) + .25))
        ax.set_title(f"{name} {coordinates(p)} - {status}", fontsize=11)
        ax.set_ylabel("Барицентрический вес")
        ax.grid(axis="y", alpha=.18)
        ax.set_axisbelow(True)
    for ax in list(axes.flat)[len(points):]:
        ax.set_visible(False)
    fig.suptitle("Метод 4 - решение системы и веса A, B, C, D", fontsize=15)
    fig.text(.5, .02, "Сумма весов равна 1. Отрицательный вес: точка снаружи.",
             ha="center", fontsize=11)
    fig.subplots_adjust(left=.08, right=.97, top=.90, bottom=.09,
                        hspace=.55, wspace=.30)
    return fig
```

A12 Параметры запуска и точка входа

```
def main():
    parser = argparse.ArgumentParser(
        description="Четыре метода проверки точки и визуализация")
    parser.add_argument("--point", nargs=3, type=float, metavar=("X", "Y", "Z"),
                        help="добавить свою точку P и выбрать её для четырёх объёмов")
    parser.add_argument("--no-plot", action="store_true",
                        help="не открывать графические окна")
    parser.add_argument("--save-dir", type=Path,
                        help="сохранить пять рисунков PNG в эту папку")
    args = parser.parse_args()
    points = list(TEST_POINTS)
    if args.point is not None:
        points.append(("P", tuple(args.point)))
    if not points or len({name for name, p in points}) != len(points):
        parser.error("Нужна хотя бы одна точка; имена точек должны быть уникальными")
    for point in list(VERTICES) + [p for name, p in points]:
        if len(point) != 3 or not all(isfinite(x) for x in point):
            parser.error("Каждая точка должна содержать три конечных числа")
    try:
        print_results(VERTICES, points)
    except ValueError as error:
        parser.error(str(error))

    if args.no_plot and args.save_dir is None:
        return
    try:
        import matplotlib
        if args.no_plot:
            matplotlib.use("Agg")  # Сохранение PNG без графического экрана.
        import matplotlib.pyplot as plt
    except ImportError:
        parser.exit(1, "\nДля рисунков установите библиотеку:\n"
                       "python -m pip install matplotlib\n"
                       "Для одних расчётов используйте --no-plot.\n")
    selected = "P" if args.point is not None else points[0][0]
    overview, parts_figure, selector = create_plots(VERTICES, points, selected)
    figures = [
        ("tetrahedron_points.png", overview),
        ("tetrahedron_volumes.png", parts_figure),
        ("tetrahedron_signs.png", create_signs_plot(VERTICES, points)),
        ("tetrahedron_planes.png", create_planes_plot(VERTICES, points)),
        ("tetrahedron_barycentric.png",
         create_barycentric_plot(VERTICES, points)),
    ]
    if args.save_dir is not None:
        args.save_dir.mkdir(parents=True, exist_ok=True)
        for filename, figure in figures:
            figure.savefig(args.save_dir / filename, dpi=180)
        print(f"\nРисунки сохранены в: {args.save_dir.resolve()}")
    if not args.no_plot:
        print("\nОткрываются пять окон. Во втором есть переключатель точек.")
        plt.show()
    else:
        for _, figure in figures:
            plt.close(figure)


if __name__ == "__main__":
    main()
```